Documentation

Generated with CodeDocTools (https://github.com/borjafdezgauna/CoderDocTools)

1 _Sidebar

Menu

HOME

Step by step - from zero to hero

- Overview

- Step 0: Blockchain Basics

- Step 1: SpaceXpanse Daemon <!-- <= start here Using the two-em dash here - U+2E3A -->

- Step 2: The Game State Processor

- Step 3a: libspex Component Relationships

- Step 3b: Compile libspex in Windows

- Step 3b: Compile libspex in Ubuntu

- Step 4: Run SpaceXpanse Daemon for Games

- Step 5: Hello World! in C++

- Step 5: Hello World! in C#

- Step 5: Hello World! with SQLite

<!-- * Step 6a: Mover Overview --

- Step 6b: Mover Console

- Step 6c: Mover Unity

-->

<!-- * Prerequisites -->

SpaceXpanse ROD blockchain

- Overview

- Tech Specs

- Blockchain Consensus Protocol

- Dual Algo Mining

- Currencies/Tokens

- Daemon Interface

- Games/dApps

- Atomic Trading

ROD core wallet

- Build ROD core wallet

- Data Directory

- spacexpanse.conf

- Daemon Options

RPC Methods

- SpaceXpanse RPC Methods

- Quick List

- Complete Reference

Tools

- SpaceXpanse QT Toolkit

- SpaceXpanse Daemon

- SpaceXpanse Daemon CLI

libspex C++ library

- How to Compile libspex in Ubuntu 20.04.03

- How to Compile libspex in Ubuntu 22.04

- How to Compile libspex in Windows

- List of Dependencies in Window

- libspex Component Relationships

SpaceXpance Metaverse Simulator

- Documentation

- Releases

Downloads

Wallets

- ROD core wallets

<!-- * Get a Wallet

- SpaceXpanse Electron wallet help

-->

Other

- SpaceXpance Metaverse Simulator

- Downloads for developers

- Tutorials Code

Tutorials

- Getting Started with Regtest

- How to Wire Up libspex in C#

- RPC Windows C# Tutorial

- Atomic Transactions Tutorial

Hello World!

- Hello World! C++

- Hello World! C#

- Hello World! with SQLite

Mover Game

- Mover Sample Game Overview

- Mover Console Tutorial

- Mover Sample Game in C# with Unity

Ships

- Ships (How to get started playing)

Tutorials Code

<!-- #### Callbacks

- forwardCallbackResult and Processing Moves -->

Miscellaneous

- FAQs

- Game Engines

- Sample in C# with Unity

- Get Some RODs

- ROD Mining

To top

2 First Steps

2.1 First Steps Tutorial

There are many tutorials and documents to help you get started, but let's get straight to some fast action by learning about SpaceXpanse and specifically spacexpansed, the daemon engine at the core of the SpaceXpanse blockchain!

The SpaceXpanse blockchain is a decentralised data-store.

For minimal fees, people can create unique names (also called accounts) in the SpaceXpanse blockchain and update them with additional data (called values). The main attribute is that only the person who created a name has its private key to update this additional data. They cannot be stolen, forged or censored.

Values for names can be anything up to 2,048 bytes. For example, you could create a name called "Rita" and attach your email address to it. Then anyone who knows that you own "Rita" can see it on the SpaceXpanse blockchain and trust the value. In this case that is your email address, but you could store other data as well.

On SpaceXpanse these names are also known as accounts and the values (the data attached to the name) are moves in a game. Only the owner of the account can make moves from that account as they own the private key.

With just that above, it's possible to build decentralised games on SpaceXpanse.

For example, imagine Rita and Sue both have their own names (accounts) stored in the blockchain. They want to play a game of chess and they decide that Rita is white (starts first).

Rita can update the value of her account with the first move:

"e2 to e4"

After about 30 seconds this will enter the blockchain.

Now Sue (or anyone in the world) can read Rita's move in the SpaceXpanse blockchain and know it came from Rita. Sue would then update the value of her account with her own move. The game would continue back and forth until someone won.

This allows for a fully decentralised game of chess that anyone can verify and see who the winner is or if they tried an invalid move (tried to cheat).

While that example is very simple, it demonstrates the fundamentals of how games on SpaceXpanse work.

In this first steps tutorial we'll explain the steps to create your own unique secure name on the SpaceXpanse blockchain and update its value.

2.2 Get a Wallet

To interact with the SpaceXpanse blockchain you will need to run a SpaceXpanse wallet. There are multiple wallets but in this guide we will use spacexpansed which is the headless daemon with an RPC interface.

If you haven't already, download one of the ZIP files then extract it.

2.3 Run the Daemon (spacexpansed)

Open a command prompt or terminal and navigate to the wallet folder.

Run the following command:

spacexpansed -rpcport=11998 -rpcuser=user -rpcpassword=password

The blockchain will begin synchronising (downloading) the blockchain. This can take some time and largely depends on your computer and network connection.

When spacexpansed first runs it downloads all the blocks from peers until it catches up to the current block that everyone is on. If you stop the daemon and restart it, it will continue from where it left off. This could take some time depending on the speed of your computer, your internet connection, and your hard disk.

Open up another command prompt or terminal and navigate again to the spacexpansed folder. You'll use this window to issue commands.

2.4 spacexpanse-cli and curl

Now that we have the daemon running, we'll use the SpaceXpanse Command Line Interface (spacexpanse-cli) or curl to issue commands. It's very easy and you'll learn along the way. For SpaceXpanse, you'll find that spacexpanse-cli is far easier than curl. The spacexpanse-cli program is included in the wallet download mentioned above.

However, there are many ways to interact with an RPC interface, and all of the ones that work for Bitcoin should also work for SpaceXpanse. Here's a link to some code examples).

2.5 Check the Network

Your first task is to check the network by confirming you are connected to other p2p spacexpanse nodes. Here are the spacexpanse-cli and curl commands:

spacexpanse-cli -rpcuser=user -rpcpassword=password getnetworkinfo

Or with curl:

curl --user user:password --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "getnetworkinfo", "params": [] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

NOTE: With both spacexpanse-cli and curl you will need to ensure that you've properly escaped anything that requires it. Also, mind which quotes you use (' vs "), depending on your platform.

Those commands will return something similar to this:

{

"version": 1019900,

"subversion": "/SpaceXpanse:1.1.99/",

"protocolversion": 110015,

"localservices": "000000000000040d",

"localrelay": true,

"timeoffset": 1,

"networkactive": true,

"connections": 8,

"networks": [

{

"name": "ipv4",

"limited": false,

"reachable": true,

"proxy": "",

"proxy_randomize_credentials": false

},

{

"name": "ipv6",

"limited": false,

"reachable": true,

"proxy": "",

"proxy_randomize_credentials": false

},

{

"name": "onion",

"limited": true,

"reachable": false,

"proxy": "",

"proxy_randomize_credentials": false

}

],

"relayfee": 0.00100000,

"incrementalfee": 0.00001000,

"localaddresses": [

],

}

From here we can see that we are connected to 8 nodes. Which is the set maximum number of outgoing connections in the client. It's possible to open ports on your system to receive incoming connections, but this is not required. You can ignore the other information in there for this tutorial.

If you don't have any connections then you may need to wait a few minutes or you may have some networking issues on your computer, so you should sort that out before continuing.

2.6 Check Synchronisation

Next, lets check how far we are with syncing. Type in the following command.

spacexpanse-cli -rpcuser=user -rpcpassword=password getblockchaininfo

Or with curl:

curl --user user:password --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "getblockchaininfo", "params": [] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

You'll get a result similar to this:

{

"chain": "main",

"blocks": 598466,

"headers": 598466,

"bestblockhash": "4ff74462ecde7624ed069687954da31cec2c3fc3328a669b7a62511a5ad7753d",

"mediantime": 1550319823,

"verificationprogress": 0.9999955845690718,

"initialblockdownload": false,

"chainwork": "0000000000000000000000000000000000000000000001a29ea874e394c2041f282",

"size_on_disk": 567360856,

"pruned": false,

"softforks": [

{

"id": "bip34",

"version": 2,

"reject": {

"status": true

}

},

{

"id": "bip66",

"version": 3,

"reject": {

"status": true

}

},

{

"id": "bip65",

"version": 4,

"reject": {

"status": true

}

}

],

"bip9_softforks": {

},

}

The value for blocks is the current block your client is synced with. The headers value is the current block number of the headers downloaded. Generally if you are still syncing, headers is greater than blocks. Once you are fully synced, blocks = headers.

You can check if you are fully synced by checking the block explorer, which should be on the latest block.

On Linux you can use the following command to see the blocks come in in real time.

watch spacexpanse-cli -rpcuser=user -rpcpassword=password getblockchaininfo

2.7 ROD Powers the SpaceXpanse Blockchain

ROD is the native cryptocurrency for the SpaceXpanse blockchain. It powers transactions. Operations such as updating a name (see below) require tiny amounts of ROD. That small amount of ROD is a fee charged by miners who secure the blockchain.

2.8 Get a ROD Address

You'll need some ROD for some commands, but you'll need a ROD address first. ROD addresses are used to send and receive coins. Let's get you a ROD address. Issue the following command. You can change the label for the address, "This is my new address!", to anything you like.

spacexpanse-cli -rpcuser=user -rpcpassword=password getnewaddress "This is my new address!"

Or with curl:

curl --user user:password --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "getnewaddress", "params": [] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

The daemon will return a single ROD address. It will look similar to the following.

RSdTuozaJbMsEjha54AbN6AFHSn4uT9nMX

Copy and save the address that the daemon returned. You can use it to receive ROD now. (Do not use the example address above.)

2.9 Get Some ROD

Our examples use mainnet, which requires real ROD. For this tutorial you'll need less than 0.1 ROD. Developers can ask for that ROD in our Discord and navigate to rod-airdrop-giveaway channel. You need only provide us with a valid ROD address. There're other means to obtain ROD coins too.

However, you can also do this on testnet or regtestnet for free. See Getting Started with Regtestnet for more information about that.

2.10 Get a Balance

Now that you have some ROD, let's check your balance. Try out the following command.

spacexpanse-cli -rpcuser=user -rpcpassword=password getbalance

Your balance will be returned similar to this:

1.23456789

2.11 What is a SpaceXpanse Name?

Before we create a name, it would be useful to know what a name is. The short answer is that it's a unique ID that only the owner can control. Let's find out what that means.

From the SpaceXpanse Specifications blockchain document:

<blockquote>

Name and Value Restrictions

Valid names and values in SpaceXpanse must satisfy additional constraints

compared to Namecoin, which only enforces maximum lengths. In particular,

these conditions must be met by names and values:

- Names can be up to 256 bytes long and values up to 2,048 bytes.
- Names must have a namespace. This means that they must start with

  one or more lower-case letters and /. Expressed as a regexp, this

  means they must match: [a-z]+\/.*
- Names must be valid UTF-8 and must not contain unprintable ASCII characters,

  i.e. those with a value less than 0x20.
- Values must be valid JSON and parse to a JSON object.

</blockquote>

If you own a name, you can update its value. This functions as a key/value pair, where the name is the key. The value can be used inside of games, for ID purposes, among other things as well.

The p/ namespace is used for game accounts, i.e. "p" is for "player".

The g/ namespace is used for game accounts, i.e. "g" is for "game".

But this is subject to change in some of the next versions!

The new namespaces can be `user/` and/or `u/` reserved for usernames, `dapp/` and/or `d/` for decentralized applications, `site/` and/or `s/` for decentralized websites through planned dDNS functionality and `meta/` and/or `m/` for metaverses.

Some examples of valid names include:

- p/Rita
- p/Sue
- g/helloworld

- g/mv

2.12 Create a SpaceXpanse Name

Names are the core of SpaceXpanse. Only YOU can control your name and do moves in games. Creating names is very easy.

In your command prompt, issue a command like this where "\<my name\>" is your chosen name:

spacexpanse-cli -rpcuser=user -rpcpassword=password name_register "p/<my name>" "{}"

Or with curl:

curl --user user:password --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "name_register", "params": [{"p/<my name>"}] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

<!-- check that curl -->

You should receive a response as a transaction ID, which is a long hexadecimal value, e.g.:

28d489d22ec3908978ce678746ad270a60254d8edd4a978d286d1251ebfb8043

If it fails, most likely you've tried to get a name that is already reserved. Check the error message and try again.

Congratulations! You've immortalised yourself on the SpaceXpanse blockchain!

2.13 Is Your Name in the Blockchain Yet?

Now that you have your first name, let's see if it's in the blockchain. Try the following command:

spacexpanse-cli name_pending "p/<my name>"

Or with curl:

curl --user user:password --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "name_pending", "params": [] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

If it returns a result, your name hasn't been mined into the blockchain yet. Wait a minute then try again.

2.14 Find All Your Names

You can find all the names you own in your wallet. Try the following command:

spacexpanse-cli name_list

Or with curl:

curl --user myusername --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "name_list", "params": [] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

That returns a list of all your names.

2.15 Update the Value for Your Name

Now that you have a name, it's time to say hello! by updating it's value (aka submitting a move)

Try running the following command with the name you registered above.

spacexpanse-cli -rpcuser=user -rpcpassword=password name_update "p/<my name>" "{\"g\":{\"helloworld\":{\"m\":\"Hello everybody!\"}}}"

Or with curl:

curl --user user:password --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "name_update", "params": ["p/<my name>", "new-value"] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

Congratulations! You just made a move in the "Hello World!" game! In a later tutorial you'll be able to see your move along with everyone else's moves and we'll explain the JSON object that is required for values on the chain.

Note how the value is escaped with back slashes.

2.16 View your name in the Blockchain

Now you or anyone in the world can lookup your name on the spacexpanse blockchain. Try running the following command with your name.

spacexpanse-cli -rpcuser=user -rpcpassword=password name_show "p/<my name>"

Or with curl:

curl --user user:password --data-binary '{"jsonrpc": "1.0", "id":"curltest", "method": "name_show", "params": ["p/<my name>"] }' -H 'content-type: text/plain;' http://127.0.0.1:11998/

you should see some information about your name along with the value you set for it.

Congratulations! You've completed your first steps.

3 How To Compile Libspex in Windows

3.1 How to Compile libspex

In this tutorial, we'll compile libspex for C++ so that we can use it in other tutorials and even in our own games.

MSYS2

Download MSYS2 x86_64 (https://www.msys2.org/) from this link:

https://github.com/msys2/msys2-installer/releases/download/2022-01-28/msys2-x86_64-20220128.exe

Install into default path C:/msys64 then run MSYS2 MingGW 64-bit (NOT the 32-bit version) from your Start menu.

Update with pacman as follows.

pacman -Syuu

Type 'Y' to close the terminal and restart MSYS2 MingGW 64-bit again.

Build & Install libspex

Next, run this command to get and run the script that will build libspex for you.

curl -o ./build.sh https://raw.githubusercontent.com/SpaceXpanse/Documentation/main/Code/libspex/Compile-scripts/build.sh; ./build.sh

You may need to press Enter and enter Y 1 or more times for the build to begin.

Done!

CONGRATULATIONS!

Congratulations! You've just built your own GSP using libspex. You can now proceed on to the Hello World in C++ tutorial where we'll put libspex to good use!

Building Lite Mode support files

Run this command to get and run the script that will build lite mode libraries for you.

curl -o ./buildLiteMode.sh https://raw.githubusercontent.com/SpaceXpanse/Documentation/main/Code/libspex/Compile-scripts/buildLiteMode.sh; ./buildLiteMode.sh

Update libspex

We update libspex periodically. If you wish to update your build, enter the following into the same MSYS2 MingGW 64-bit terminal that you used above.

cd ~/libspex

git pull

make clean

./autogen.sh

./configure

make -j2

make install

You're now up-to-date with the latest version!

4 How To Compile Libspex in Ubuntu

4.1 How to Compile libspex in Ubuntu 20.04.3

This will help you to compile libspex in Ubuntu so you can use it in other tutorials and in your own games.

Most dependencies can simply be installed, but some you will need to build and install from source.

You can optionally download the install-libspex.sh script and run it. It runs the same commands as below, but will prompt you to continue at each new stage.

Prepare to Build and Install libspex

The following build instructions assume a fresh installation of Ubuntu 20.04.3 LTS.

NOTE: For the most current information, refer to the libspex repository here:

https://github.com/spacexpanse/libspex

Update and Upgrade Ubuntu

sudo apt-get -y update

You can upgrade, but it's not required, and was not used for this build.

sudo apt-get upgrade

Install Prerequisites & Dependencies with apt

The following will install most dependencies for you. Some others must be built from source.

sudo apt-get -y install build-essential libargtable2-dev libzmq3-dev zlib1g-dev libsqlite3-dev liblmdb-dev libgoogle-glog-dev libgflags-dev libprotobuf-dev protobuf-compiler python3 python-protobuf autoconf autoconf-archive automake cmake git libtool pkg-config libcurl4-openssl-dev libssl-dev libmicrohttpd-dev make

Build and Install Bitcoin-Core libsecp256k1

The Bitcoin-Core libsecp256k1 library is an optimised C library for ECDSA signatures and secret/public key operations on curve secp256k1.

cd ~/

git clone https://github.com/bitcoin-core/secp256k1.git

cd secp256k1

./autogen.sh

./configure --disable-tests --disable-benchmark \

--enable-module-recovery

make

sudo make install

Build Googletest from Source

googletest must be built from source.

cd ~/

git clone https://github.com/google/googletest

cd googletest

cmake .

make -j4

sudo make install/strip

Build and Install SpaceXpanse Eth-Utils

The SpaceXpanse Eth-Utils library is a small C++ library that implements basic building blocks for working with Ethereum from C++.

cd ~/

git clone https://github.com/spacexpanse/eth-utils.git

cd eth-utils

./autogen.sh

./configure

make -j4

sudo make install

Build and Install JSONCPP from Source

libspex requires JSONCPP v1.7.5 or higher, but only v1.7.4 is available to install via apt.

Further, the build process is somewhat odd, so make certain to follow the instructions below, or ensure that any changes you make don't break the libspex build process.

Specifically, v1.9.4 has a broken pkg-config file which must be specifically fixed. The following is currently a "hack" until a new version is released. You can refer to https://github.com/spacexpanse/libspex/blob/ubuntu-build/Dockerfile for version updates.

cd ~/

mkdir jsoncpp && cd jsoncpp

git clone -b 1.9.4 https://github.com/open-source-parsers/jsoncpp .

git config user.email "test@example.com"

git config user.name "Cherry Picker"

git cherry-pick ac2870298ed5b5a96a688d9df07461b31f83e906

cmake . -DJSONCPP_WITH_PKGCONFIG_SUPPORT=ON -DBUILD_SHARED_LIBS=ON -DBUILD_STATIC_LIBS=OFF

make -j4

sudo make install/strip

Build JSON-RPC-CPP from Source

JSON-RPC-CPP must be built from source because the available packages are not fresh enough. Specifically, they must include commit 4f24acaf4c6737cd07d40a02edad0a56147e0713.

cd ~/

git clone https://github.com/cinemast/libjson-rpc-cpp

cd libjson-rpc-cpp

cmake . -DREDIS_SERVER=NO -DREDIS_CLIENT=NO -DCOMPILE_TESTS=NO -DCOMPILE_EXAMPLES=NO -DWITH_COVERAGE=NO

make -j4

sudo make install/strip

Install ZMQ C++ Bindings from Source (CPPZMQ)

The following will install the ZMQ C++ bindings.

cd ~/

git clone -b v4.7.1 https://github.com/zeromq/cppzmq

cd cppzmq

sudo cp zmq.hpp /usr/local/include

Clean Up and Make All Installed Dependencies Visible

The following is a clean-up step and will make our installed dependencies visible for when we build libspex.

sudo ldconfig

Build libspex

With the above accomplished, building and installing libspex is simple and straight forward.

cd ~/

git clone https://github.com/spacexpanse/libspex.git

cd ~/libspex

./autogen.sh

./configure

make -j4

sudo make install

Clean Up and Make libspex Visible

Similar to the clean-up step above, this does the same and will make libspex visible for when you write your GSP.

sudo ldconfig

CONGRATULATIONS!

Congratulations! You've just built and installed libspex. You can now proceed on to the Hello World in C++ tutorial where we'll put libspex to good use!

Update libspex

We update libspex periodically. If you wish to update your build, enter the following into a terminal.

cd ~/libspex

git pull

make clean

./autogen.sh

./configure

make -j4

sudo make install

sudo ldconfig

You're now up-to-date with the latest version!

NOTE: Should the update build break, refer to the libspex repository as it is always up-to-date:

https://github.com/spacexpanse/libspex

You can also refer to the libspex Docker build file for further specifics:

https://github.com/spacexpanse/libspex/blob/ubuntu-build/Dockerfile

5 Running Spacexpansed For Games

5.1 Running spacexpansed for Games

It's important to set the right options when you run spacexpansed so that you can communicate with it and so that libspex can also communicate with it.

The short answer is that you should generally run it as follows.

spacexpansed -rpcuser=user -rpcpassword=password -wallet=game.dat -server=1 -rpcallowip=127.0.0.1 -zmqpubgameblocks=tcp://127.0.0.1:28332 -zmqpubgamepending=tcp://127.0.0.1:28332

It will then run as a server.

However, you will need to supply the rpcuser ("user" above), the rpcpassword ("password" above), and the wallet for every request you make. Using spacexpanse-cli a command would then look like this:

spacexpanse-cli -rpcuser=user -rpcpassword=password -wallet=game.dat <command>

If you have specified the rpcuser, rpcpassword or wallet in spacexpanse.conf in the data directory, then you can leave them out above when running spacexpansed. You will then need to specify the rpcuser, rpcpassword and wallet from the spacexpanse.conf file when you issue commands with spacexpanse-cli or through any other RPC tool that you may use.

If you do not specify a wallet anywhere, spacexpansed will default to the wallet.dat file in the data directory.

Running Naked

You can run spacexpansed without specifying any rpcuser, rpcpassword or wallet, in which case you can leave them out entirely when you run commands through spacexpanse-cli. spacexpanse-cli commands are then much simpler.

spacexpanse-cli <command>

To run spacexpansed very simple for spacexpanse-cli and not specify any rpcuser, rpcpassword or wallet in spacexpanse.conf, run spacexpansed like so:

spacexpansed -server=1 -rpcallowip=127.0.0.1 -zmqpubhashtx=tcp://127.0.0.1:28332 -zmqpubhashblock=tcp://127.0.0.1:28332 -zmqpubrawblock=tcp://127.0.0.1:28332 -zmqpubrawtx=tcp://127.0.0.1:28332 -zmqpubgameblocks=tcp://127.0.0.1:28332

That is the minimum you must use to run spacexpansed for games, testing, or development.

See Also

See more information about spacexpanse.conf here.

See more information about spacexpanse-cli here.

See more information about SpaceXpanse RPC methods here.

See more information about daemon startup options for spacexpansed here.

6 Hello World CXX

6.1 Hello World in C++

99% of everything you need to know about creating games and other dApps on the SpaceXpanse Multiverse platform is covered in this tutorial. Only a few topics aren't covered, such as game logic.

While this is long, it is thorough and you will learn many core concepts:

1. Starting spacexpansed with the proper options

1. Starting your game daemon with the proper options

1. Working with daemons (spacexpansed and libspex)

1. Calling libspex

1. Getting a game state

1. Making a move

All of this will become clear as we build and run a concrete implementation.

In this tutorial we're going to build a Hello World console application in C++ from scratch. Once we're finished, we'll run it, check its game state, then make a move and say hello to everyone. This involves several major steps.

1. Write our Hello World code

1. Compile libspex so that we can include it in our executable

1. Compile our Hello World game with libspex

1. Sort out the extra dependency files that we need

1. Start spacexpansed and Hello World with the proper options

1. Check the game state

1. Make a move and say hello!

Our application will incorporate libspex and will run as a daemon. In another command prompt or terminal, we'll send moves to the game via JSON RPC and retrieve the results as serialised JSON. Our sending and receiving there mimics a "front end" as it were.

6.2 What Does Hello World Do?

The Hello World game that you're about to write does 1 simple thing: give people a way to say "Hello!" and then return a list of the last message that they sent. No more. No less.

6.3 Download the Code

All the code for Hello World is available for you as a finished product here. You can either write the code yourself as we progress, or you can open up the code in your editor and follow along.

All Hello World tutorial code (and precompiled dependencies for Windows) is available in the SpaceXpanse Tutorial Code repository. Please download it from there.

The repository contains some extra files/scripts to help with compilation and running.

6.4 Create helloworld.cpp and Add Includes

Open up your text editor and create a helloworld.cpp file. Next, add in these includes:

```
#include <spacexpansegame/defaultmain.hpp>
#include <gflags/gflags.h>
#include <glog/logging.h>
#include <json/json.h>
#include <cstdlib>
#include <iostream>
#include <sstream>
```

"spacexpansegame/defaultmain.hpp" adds in libspex. This is the core daemon that processes game states for you.

The "gflags/gflags.h" and "glog/logging.h" includes are for command line option processing and the Google logging module. You can find them here and here, respectively. Don't get hung up on these though. They're just libraries that we'll be using to make our lives easier.

We've chosen to use JSON RPC, and "json/json.h" gives us that.

The other includes are simply standard libraries.

6.5 Create an Anonymous Namespace

Our game logic and connection variables will be in an anonymous namespace, so go ahead and create that.

```
namespace { }
```

Now we can add code to our anonymous namespace.

6.6 Define Connection Variables

Hello World will run as a daemon, and in order for us to connect to our Hello World daemon, we need several options. We'll pass these options to the daemon as command line arguments when we run it.

- spacexpanse_rpc_url: URL at which SpaceXpanse Core's JSON-RPC interface is available

- game_rpc_port: The port at which the game daemon's JSON-RPC server will be start (if non-zero).

- enable_pruning: If non-negative (including zero), enable pruning of old undo data and keep as many blocks as specified by the value

- storage_type: The type of storage to use for game data (memory, sqlite or lmdb)

- datadir: The base data directory for game data (will be extended by the game ID and chain); must be set if --storage_type is not memory

We're using the Google Flags library (gflags) for these as it greatly simplifies our code. It provides various "DEFINE_xxx" methods. Each has 3 parameters:

1. Variable name

1. Variable value

1. Comment about the variable, i.e. documentation

Go ahead and create the variables listed above as shown below. You can copy and paste that into your helloworld.cpp file under the includes.

```
DEFINE_string (spacexpanse_rpc_url, "http://127.0.0.1:11999", "");
DEFINE_int32 (game_rpc_port, 29050, "");
DEFINE_int32 (enable_pruning, -1, "");
DEFINE_string (storage_type, "memory", "");
DEFINE_string (datadir, "", "");
```

Next, change spacexpanse_rpc_url and game_rpc_port if required by your environment. Keep in mind that we'll be using those values throughout this tutorial, so if you change them, you must maintain consistency.

Leave the others as they are. We aren't using a data directory in this example, so blank is fine.

6.7 Create the HelloWorld Class

Our HelloWorld class contains all the game logic. In our main method, we'll create an instance of HelloWorld and pass that to libspex. libspex will understand everything in HelloWorld, process it, and return values for us, i.e. new game states.

The HelloWorld class inherits from spacexpanse::CachingGame. CachingGame is an easy way to create simple games. Its main advantage is that we don't need to create any undo data, which we'll look at in a future tutorial. Let's add the HelloWorld class now.

```
class HelloWorld : public spacexpanse::CachingGame
{
    protected:
}
```

We're going to add 3 methods to this class.

1. GetInitialStateInternal: Sets the starting point on the blockchain

1. UpdateState: Receives the previous game state and updates it

1. GameStateToJson: A simple helper method

6.8 Write the GameStateToJson Method

GameStateToJson is the simplest method, so let's add it to our HelloWorld class first.

While we may not strictly need this method in our Hello World game, it's nice to have. Copy and paste it into our HelloWorld class.

```
GameStateToJson (const spacexpanse::GameStateData& state) override
{
    std::istringstream in(state);
    Json::Value jsonState;
    in >> jsonState;
    return jsonState;
}
```

The GameStateToJson method overrides the GameStateToJson method in libspex's spacexpanse::CachingGame class.

In there we get our GameStateData into a string buffer, then store it in a JSON value and return the JSON. This makes dealing with our GameStateData easier to handle because it's now just a big bunch of key/value pairs stored in JSON.

6.9 Write the GetInitialStateInternal Method

Next, our GetInitialStateInternal is similarly quite easy. It decides which chain to run on and which block to start at. It's only ever used once, but it's critical to get it right. We don't want to start at block 1 because then we need to look at every block before the game starts.

Start writing that method as shown below.

```
spacexpanse::GameStateData
GetInitialStateInternal (unsigned& height, std::string& hashHex)
override
{ }
```

Again, GetInitialStateInternal overrides the GetInitialStateInternal method in libspex's spacexpanse::CachingGame similar to how GameStateToJson did above.

Choose Which Chain to Run On

In GetInitialStateInternal we'll decide which chain our Hello World game will run on, i.e. mainnet, testnet, or regtest. We'll use libspex's spacexpanse::Chain enumeration to choose. Add in a switch case block as shown below.

```
switch (GetChain ())
{
    case spacexpanse::Chain::MAIN:
    break;
    case spacexpanse::Chain::TEST:
    break;
    case spacexpanse::Chain::REGTEST:
    break;
    default:
    LOG (FATAL) << "Invalid chain: " << static_cast<int> (GetChain ());
}
```

The GetChain method detects which chain we're on, i.e. mainnet, testnet, or regtest. We'll actually decide what chain we want to run on at runtime when we pass in a command line argument for spacexpanse_rpc_url. Here are some possibilities for mainnet, testnet, and regtest, respectively:

- --spacexpanse_rpc_url="http://user:password@localhost:11999"

- --spacexpanse_rpc_url="http://user:password@localhost:18399"

- --spacexpanse_rpc_url="http://user:password@localhost:18499"

With that decided, we set the height and hashHex values.

The height is the block height that we want our game to start at. This should be the highest possible block height that is prior to any moves being made in our game. There's no sense in processing blocks with no game data in them.

The hashHex is the block hash for that block. You can look up the block hash at http://explorer1.rod.spacexpanse.org:3001/. We're going to start our game at block 555,555.

Go ahead and add that to the mainnet case as shown below.

```
case spacexpanse::Chain::MAIN:
    height = 555555;
    hashHex


= "8dfc61ce31adcc1b3e4a02148e9ef8dfdd09dfac3b0b7aebfea30a3c8be6819d";
    break;
```

You can fill in the values for testnet and regtest or leave them blank if you wish as we won't be using them. In a real game, you would have to fill in those values because you would need to use testnet and regtest during development. We'll look at testnet and regtest in other tutorials.

Since the initial state is only run once, and we need to have a valid value for our GameStateData, return a simple, valid JSON string after the switch case block as shown below.

```
return "{}";
```

That's the end of GetInitialStateInternal. We now turn our attention to the meaty goodness of processing "moves" in the game.

6.10 Write the UpdateState Method

The UpdateState method iterates over the various players in our game and creates a game state. Game states are representations of what the game world looks like at a particular "time" or block height. (See Time on the Blockchain for more information about that.)

Wire up your UpdateState method as shown below.

```
spacexpanse::GameStateData
UpdateState (const spacexpanse::GameStateData& oldState,
     const Json::Value& blockData) override
{ }
```

Notice again that UpdateState overrides the UpdateState method in libspex's spacexpanse::CachingGame.

Get the Previous Game State

We need 1) the game state from the previous block, and 2) the new moves in order to process our game logic and create a new game state. We'll just use the new moves raw as they're already delivered to us as a Json::Value&.

Get the previous game state (oldState), store it in a string buffer, and then put it into a Json::Value so that we can use it easily.

Type or copy and paste the following into your UpdateState method.

```
std::istringstream in(oldState);
Json::Value state;
in >> state;
```

We now have our previous game state stored in state.

Iterate Over the Moves

We need to look at all the new moves in the new block data. Our blockData will contain JSON similar to the following.


```
{
   "block": {
    "hash":
"dda7eccde4857742e5000bd66cf72154ce26c22876582654bc8b8d78dadbce8c",
    "height": 558369,
    "parent":
"18f72c91c7b9223e9c7d0525216277e4016d748a2c81be4ba9d4a2b30eaed92d",
    "rngseed":
"b36747498ce183b9da32b3ab6e0d72f2a17aa06859c08cf1d1e91907cb09dddc",
    "timestamp": 1549056526
   },
   "moves": [
    {


"move": {
          "m": "Hello world!"


},


"name": "ALICE",


"out": {
          "CMBPmRos5QADg2T8kvkQhMaMV5WzpzfedR": 3443.7832612


},


"txid":
"edd0d7a7662a1b5f8ded16e333f114eb5bea343a432e6c72dfdbdcfef6bf4d44"
    }
   ],
   "reqtoken": "1fba0f4f9e76a65b1f09f3ea40a59af8"
}
```


The "moves" node is an array. We are only interested in the "move" and the "name".

Go ahead and wire up a for statement to iterate over all the "moves" in our blockData.

```
for (const auto& entry : blockData["moves"])
{
}
```

As mentioned above, we're really only interested in "name" and "move". Let's get them from our blockData into a string and an auto&.

```
const std::string name = entry["name"].asString ();
const auto& mvData = entry["move"];
```

Check for Errors

The "name" is simple enough, but we don't know anything about what the "move" is.

We know what a valid move should look like, but we don't know what THIS move is, so we must do some error checking.

Check to see if we have a valid object. If it's not valid, then we log ourselves a message and go back to the top of our for loop.

Go ahead and add some code to do that or copy and paste the following into your UpdateState method.

```
if (!mvData.isObject ())
{
    LOG (WARNING)
          << "Move data for " << name << " is not an object: " <<
mvData;
    continue;
}
```

Get the Hello World Message

At last we've reached the point where we can get the actual messages that people are sending.

If you recall from the JSON above, our "move" data looks like this:

```
"move": {
         "m": "Hello world!"
         }
```

So we can access the message through "m".

Go ahead and store it in a variable.

```
const auto& message = mvData["m"];
```

Check the Message for Errors

Anyone can submit a move into the SpaceXpanse blockchain, and there's no guarantee that a move will be valid. Let's check to make certain that we have a string and not something else.

Go ahead and write some error checking code, or copy and paste the following.

```
if (!message.isString ())
   {
    LOG (WARNING)
          << "Message data for " << name << " is not a string: " <<
message;
    continue;
   }
```

Next, we'll update the game state.

Update the Game State

At this point, we know that we have a valid string. We now use the player's name as a key for our game state (state) and we assign its value as our message from above.

```
state[name] = message.asString ();
```

You've just updated the game state. Time to return it.

Return the Game State

With our game state completely updated, we can return it. Create an output string buffer to store our game state in, and then return it as a string.

Write that code on your own or copy and paste the following.

```
std::ostringstream out;
out << state;
return out.str ();
```

CONGRATULATIONS! We're finished our HelloWorld class and can move on to main method!

6.11 Write the main Method

Our main method will be written outside of the anonymous namespace. Wire it up as usual.

```
int main (int argc, char** argv)
{ }
```

Set Logging

If you remember the glog include way up above, we're going to start using that now. Add logging as shown below.

```
google::InitGoogleLogging (argv[0]);
```

argv[0] is the FLAGS_spacexpanse_rpc_url URL, so glog will send output to libspex. (Remember from above that we're going to compile libspex directly into our Hello World game.)

Set Flags

We must also set our flags. If you remember from above, we included gflags. We'll use that to parse command line arguments into our flags, i.e.:

- FLAGS_spacexpanse_rpc_url

- FLAGS_game_rpc_port

- FLAGS_enable_pruning

- FLAGS_storage_type

- FLAGS_datadir

You can do that manually, or you can use gflags. Go ahead and write code to parse command line arguments for our flags, or copy and paste in the following code.

gflags::SetUsageMessage ("Run HelloWorld game daemon");

gflags::SetVersionString ("1.0");

gflags::ParseCommandLineFlags (&argc, &argv, true);

The flags we listed above are now populated with the appropriate values.

Check for Errors

We must check for errors in our flags.

We must have a correct RPC URL. You can write code to do thorough error checking or copy and paste the following check into your main method.

```
if (FLAGS_spacexpanse_rpc_url.empty ())
{
   std::cerr << "Error: --spacexpanse_rpc_url must be set" <<
std::endl;
   return EXIT_FAILURE;
}
```

libspex can use 3 different types of storage:

- Memory

- SQLite

- lmdb

Memory doesn't require a data directory, but the other 2 do. Let's check for an error there. The strings for each are as above, but lower case.

Write some code to check or copy and paste the following code into your main method.

```
if (FLAGS_datadir.empty () && FLAGS_storage_type != "memory")
{
   std::cerr << "Error: --datadir must be specified for non-memory
storage"
               << std::endl;
   return EXIT_FAILURE;
}
```

Wire Up and Set a Daemon Configuration

libspex expects a daemon configuration. Copy and paste the following into your main method.

spacexpanse::GameDaemonConfiguration config;

We'll fill the configuration with data from our flags. Write code to fill the flags or copy and paste the following into your main method.

```
config.SpaceXpanseRpcUrl = FLAGS_spacexpanse_rpc_url;
if (FLAGS_game_rpc_port != 0)
```

```
{
   config.GameRpcServer = spacexpanse::RpcServerType::HTTP;
   config.GameRpcPort = FLAGS_game_rpc_port;
}
config.EnablePruning = FLAGS_enable_pruning;
config.StorageType = FLAGS_storage_type;
config.DataDirectory = FLAGS_datadir;
```

RPC Server Type

NOTE: The configuration requires an RPC server type but we didn't have an option set for it. It is set as follows.

```
config.GameRpcServer = spacexpanse::RpcServerType::HTTP;
```

In your own game you can use TCP, i.e. spacexpanse::RpcServerType::TCP. However, in Hello World we use HTTP.

Instantiate Your HelloWorld Class

The HelloWorld class is all the game logic. It's time to put it to work. Create an instance of it now.

```
HelloWorld logic;
```

libspex Startup Checklist

We need 3 things to start libspex:

1. A daemon configuration

1. A game name

1. Game logic

We created the daemon configuration through the command line options that we parsed into flags above in Wire Up and Set a Daemon Configuration.

Our game name is "helloworld".

Our game logic is our `HelloWorld` class that we named logic.

Start libspex

Copy and paste the following at the end of your main method.

```
const int res = spacexpanse::DefaultMain (config, "helloworld",
logic);
return res;
```

Connecting to libspex is a blocking operation, so return res; will never be reached.

CONGRATULATIONS! You can now compile and run your Hello World daemon.

6.12 Compile Hello World

Compiling Hello World has quite a few requirements, and for some people this is probably the most difficult part.

Our Hello World needs libspex to work. For that, we must compile libspex.

Once that's done we can compile Hello World.

Compiling libspex

There are separate tutorials to show you how to compile libspex. They have all the scripts and instructions needed.

ON WINDOWS: Finish the How to Compile libspex in Windows tutorial then return back here. If all goes well, you can complete that tutorial in a few minutes as it is very short.

ON LINUX OR MAC OS X: Consult the How to Compile libspex in Ubuntu document for how to build libspex.

Compiling Hello World

Now that you have libspex built and available, let's compile Hello World.

Compiling Hello World on Windows

1. Open up an MSYS2 MingGW 64-bit terminal

1. Create a new folder for Hello World:

C:\msys64\home\<username>\hello world\

In MSYS2 that is:

\home\<username>\hello world\

1. Copy in all the files for it, i.e.:

- helloworld.cpp <☒ mandatory

- build.sh <☒ mandatory

- hellotest.py <☒ optional

- run-regtest.sh <☒ optional

- run-mainnet.sh <☒ optional

1. Run build.sh as follows:

./build.sh

1. Done.

Our build.sh script compiled Hello World as the "hello" executable file.

Compiling Hello World on Linux or Mac OS X

1. Open up a terminal

1. Navigate to your Hello World folder

1. Run build.sh as follows:

./build.sh

1. Done.

Trouble Shooting

If you get an error running build.sh, in your terminal run the following 2 commands, i.e. the contents of the file.

packages="libspex jsoncpp libglog gflags libzmq openssl"

g++ hello.cpp -o hello -Wall -Werror -pedantic -std=c++14 -DGLOG_NO_ABBREVIATED_SEVERITIES pkg-config --cflags ${packages} pkg-config --libs ${packages} -pthread -lstdc++fs

That will build the hello executable file for you.

Copy Dependencies

If you're on Windows, our Hello World executable file, hello, won't run quite yet. libspex is built into it, but libspex has many dependencies.

If you're on Linux or Mac OS X, you already installed all the dependencies when you built libspex above in Compiling libspex, so no further action is required from you at this point. You can skip down to Run Hello World and continue.

Copy Dependencies on Windows

libspex has a long list of dependencies. See List of Dependencies for libspex in Windows for those.

You can also find them in the "Required dependencies.txt" file. For the sake of expediency, you can run the "copy-dependencies.bat" batch file from a Windows command prompt in the Hello World build folder to copy them quickly.

With the dependencies copied you can now run Hello World.

6.13 Run Hello World

We're going to need 3 command prompts or terminals.

1. To run spacexpansed

2. To run hello

3. To execute RPC commands with spacexpanse-cli and curl

Open them up when they are required and navigate to the appropriate paths.

Run spacexpansed with Proper Options

To run spacexpansed properly configured for games like our Hello World game, run it with the following options.

- -rpcuser=YOURUSER

- -rpcpassword=YOURPASSWORD

- -wallet=YOURWALLET

- -server=1

- -listen=1

- -rpcallowip=127.0.0.1

- -zmqpubhashtx=tcp://127.0.0.1:29050

- -zmqpubhashblock=tcp://127.0.0.1:29050

- -zmqpubrawblock=tcp://127.0.0.1:29050

- -zmqpubrawtx=tcp://127.0.0.1:29050

- -zmqpubgameblocks=tcp://127.0.0.1:29050

Or, all on 1 line:

-rpcuser=YOURUSER -rpcpassword=YOURRASSWORD -wallet=YOURWALLET -server=1 -listen=1 -rpcallowip=127.0.0.1 -zmqpubhashtx=tcp://127.0.0.1:29050 -zmqpubhashblock=tcp://127.0.0.1:29050 -zmqpubrawblock=tcp://127.0.0.1:29050 -zmqpubrawtx=tcp://127.0.0.1:29050 -zmqpubgameblocks=tcp://127.0.0.1:29050

NOTE: You can change rpcuser name and the rpcpassword. Also, in this example we're using a wallet. It resides in the "YOURWALLET" folder in the data directory. If you wish to use the main wallet in the data directory folder, you can simply delete "-wallet=YOURWALLET" and spacexpansed will default to it.

In a command prompt, navigate to the folder where you have spacexpansed (it's included with the SpaceXpanse QT wallet download and in the same folder as spacexpanse-cli). Start spacexpansed as shown below. (Remember to add "./" at the beginning in Linux or OS X.)

spacexpansed -rpcuser=user -rpcpassword=password -wallet=game -server=1 -listen=1 -rpcallowip=127.0.0.1 -zmqpubhashtx=tcp://127.0.0.1:29050 -zmqpubhashblock=tcp://127.0.0.1:29050 -zmqpubrawblock=tcp://127.0.0.1:29050 -zmqpubrawtx=tcp://127.0.0.1:29050 -zmqpubgameblocks=tcp://127.0.0.1:29050

spacexpansed will immediately begin scrolling information and adding more as blocks come in.

We'll see other output from our Hello World game (technically, its from libspex) below when we get the current game state with curl.

Hello World Needs Proper Options/Flags

Our Hello World daemon needs several options/flags to be set. Recall from Set Flags above that we have 5 flags that we set:

- spacexpanse_rpc_url

- game_rpc_port

- enable_pruning

- storage_type

- datadir

We set the enable_pruning option independently of any command line options. So, we must pass in the other 4 when we run Hello World.

The value for our spacexpanse_rpc_url depends upon how we started spacexpansed. It takes this form:

http://user:password@host:port

The user and password must be set or we won't be able to do many things. Above we ran spacexpansed with "user" and "password", so those are fine in the URL above.

The host should be the local host or 127.0.0.1.

http://user:password@127.0.0.1:port

The port depends upon which chain we want to run. Recall from Choose Which Chain to Run On that the port number for mainnet, testnet and regtest are 11999, 18399 and 18499, respectively. We're going to run on mainnet.

http://user:password@127.0.0.1:11999

That completes our RPC URL.

So far to run hello, we have:

hello --spacexpanse_rpc_url="http://user:password@127.0.0.1:11999"

We set our game RPC port to 29050. This could be any free port and is completely arbitrary. This is our command so far.

hello --spacexpanse_rpc_url="http://user:password@127.0.0.1:11999" --game_rpc_port=29050

Hello World is very simple and doesn't require a database, so we set storage to "memory".

hello --spacexpanse_rpc_url="http://user:password@127.0.0.1:11999" --game_rpc_port=29050 --storage_type=memory

We don't need to set a data directory if we're not using SQLite or lmdb, but let's just set one anyways.

hello --spacexpanse_rpc_url="http://user:password@127.0.0.1:11999" --game_rpc_port=29050 --storage_type=memory --datadir=/tmp/spacexpansegame

Our options/flags to run hello are complete.

Running Hello World

Navigate in a command prompt to the hello executable.

Regtest

This is the network that you would use for development. It is here for informational purposes. You should likely run on mainnet.

Running on the regtest chain will look like this:

./hello --spacexpanse_rpc_url="http://user:password@localhost:18499" --game_rpc_port=29050 --storage_type=memory --datadir=/tmp/spacexpansegame

Or on Windows like this:

hello.exe --spacexpanse_rpc_url="http://user:password@localhost:18499" --game_rpc_port=29050 --storage_type=memory --datadir=/tmp/spacexpansegame

Mainnet

Real games run on mainnet. You should use this to see actual, real-world results.

Running on mainnet will look like this (these are the options we created above):

./hello --spacexpanse_rpc_url="http://user:password@localhost:11999" --game_rpc_port=29050 --storage_type=memory --datadir=/tmp/spacexpansegame

Or on Windows like this:

hello.exe --spacexpanse_rpc_url="http://user:password@localhost:11999" --game_rpc_port=29050 --storage_type=memory --datadir=/tmp/spacexpansegame

Real-world Hello World

Run one of those now.

Hello World on Windows:

Hello World on Linux:

The output you can see in the screenshot above happens whenever a new move is made. We didn't add this to our code, but you can add that hello daemon output as we've done in the screenshot like this:

```
// Some output is nice.
std::cout << name << " said " << message << "\r\n";
```

Put that just before you store the player's message in the game state.

CONGRATULATIONS! Hello World is running!

Our next tasks are to check the state of the game with curl and make a move with spacexpanse-cli.

Trouble Shooting

If you're not getting any results, it is likely that you do not have spacexpansed running.

See the First Steps tutorial and Running spacexpansed for Games for information on how to run spacexpansed properly.

Alternatively, you can run the SpaceXpanse Electron wallet as it is preconfigured to run with all the proper options set for games, such as our Hello World example.

6.14 A Quick Note About Front Ends

Our Hello World game is running as a daemon, and we don't have a front end or GUI.

Below we'll use curl and spacexpanse-cli to mimic a front end.

We'll use curl to display results, and we'll use spacexpanse-cli to make moves.

With what we have so far, you can easily create a real GUI for yourself using any technology or language or toolkit that you prefer. You could do it with Unreal Engine, Unity, Windows Forms, WPF, or anything. You only need to be able to send RPC commands to the hello daemon (for game states to update your GUI) and spacexpansed (to make moves).

6.15 See What People Are Saying with GetCurrentState

To get the game state and see what people are saying, we must issue an RPC command to the Hello

World daemon (technically, it's libspex that will return our results, but we built libspex into the hello daemon). We can use curl for that. Note that we must use JSON 2.0.

curl --data-binary '{"jsonrpc": "2.0", "id":"curltest", "method": "getcurrentstate"}' -H content-type: text/plain;' http://127.0.0.1:29050/

Or on Windows:

curl --data-binary "{\"jsonrpc\": \"2.0\", \"id\":\"curltest\", \"method\": \"getcurrentstate\"}" -H "content-type: text/plain;" http://127.0.0.1:29050/

Open up a new command prompt or terminal and try that now.

Our result is JSON and will be similar to the following.

{"id":"curltest","jsonrpc":"2.0","result":{"blockhash":"cd8235ac63697d20732d65bd9d49bcf8107f24d4a4bc32f83e93786dd
WORLD!","Crawling Chaos":"...Hello, ALICE!","Wile E. Coyote":"Hello Road Runner!"},"height":610662,"state":"up-to-date"}}

Or prettified:


```
{
   "id" : "curltest",
   "jsonrpc" : "2.0",
   "result" : {
     "blockhash" :
"cd8235ac63697d20732d65bd9d49bcf8107f24d4a4bc32f83e93786dd8276c6a",
     "chain" : "main",
     "gameid" : "helloworld",
     "gamestate" : {
       "ALICE" : "",
       "BOB" : "HELLO WORLD!",
       "Crawling Chaos" : "...Hello, ALICE!",
       "Wile E. Coyote" : "Hello Road Runner!"
       },
     "height" : 610662,
     "state" : "up-to-date"
     }
}
```


The chain is "main", as discussed in Choose Which Chain to Run On.

You can also see our gameid is "helloworld", just as we set it above in Start libspex.

```
const int res = spacexpanse::DefaultMain (config, "helloworld",
logic);
```

However, of most interest is our game state.

```
"gamestate" : {
       "ALICE" : "",
       "BOB" : "HELLO WORLD!",
       "Crawling Chaos" : "...Hello, ALICE!",
       "Wile E. Coyote" : "Hello Road Runner!"
       }
```

If you recall from above in Update the Game State, we added players to our game state with their name as the key and their message as the value.

```
state[name] = message.asString ();
```

Now we have those back in a nice, neat key/value pair array.

If we were to have a front end for our Hello World game, we would take that game state and process it. That might mean displaying all players in a list with their messages like this:

<name> said "<message>"

Now that we know how to get the game state, let's make a move!

6.16 Moves in Hello World

With our Hello World game now running as a daemon and libspex embedded inside it, we must use another command line or terminal to send moves to the SpaceXpanse blockchain. Go ahead and open one now.

Moves are made through JSON RPC to the SpaceXpanse daemon, i.e. to spacexpansed. There are many ways to do that, but we'll limit our methods to spacexpanse-cli and curl.

If you're not already familiar with spacexpanse-cli, see the Getting Started with spacexpanse-cli tutorial.

Each game has it's own format for sending moves. For Hello World, it's trivial. ("m" stands for "message".)

"m":"<hello message>"

However, there's a general format for all games. Here you can see the above embedded in it:

{"g":{"helloworld":{"m":"<hello message>"}}}

The "g" means the game namespace and "helloworld" is the name of the game.

Do You Have a Name?

Let's find out if you have a name in your SpaceXpanse wallet. In your command prompt/terminal, navigate to the folder with the SpaceXpanse QT wallet because spacexpanse-cli is in there. We're going to issue a name_list command.

Type or copy and paste this command (we must remember that we ran spacexpansed with a username and password):

spacexpanse-cli -rpcuser=user -rpcpassword=password name_list

If you have any names, they'll be listed. If you don't have any, go back and complete that portion of the First Steps tutorial.

Do keep in mind that we can have multiple SpaceXpanse wallets, and when we run spacexpansed, we must specify which wallet we want to use if we don't want to use the default wallet. In this tutorial we're using the game wallet. For more information about wallets, consult the Wallet topic in the SpaceXpanse Electron wallet help and the data directory document.

Time to Make Your Move!

Moves are made through the name_update RPC method. The following is an example that updates your name to have no value.

spacexpanse-cli -rpcuser=user -rpcpassword=password name_update "p/<my name>" "{}"

Combining that with our move structure from above yields the following.

spacexpanse-cli -rpcuser=user -rpcpassword=password name_update "p/<my name>" "{\"g\":{\"helloworld\":{\"m\":\"Hello World!\"}}}"

Replace your name in that command and run it to make a move in our Hello World game.

You will receive an error code if you made a mistake, such as using a name that you don't own in your wallet.

If all went well you'll receive a transaction ID (or txid as is more commonly used).

> BASH ERRORS: On Linux, when running the above spacexpanse-cli command to update a name value, you may run into a bash: !\: event not found error. This is because ! has a special meaning for bash.

>

> You can turn of the bash history by running set +H and then run the name_update operation normally.

If you watch your spacexpansed command prompt, you'll see that spacexpansed saw the name_update transaction and is processing it.

CONGRATULATIONS!

You made a move in Hello World!

Now, check the game state as you did above. You may need to wait a minute until it's mined into the blockchain. (Miners provide an invaluable service.)

And we getcurrentstate from our hello daemon with curl just as we did above under See What People Are Saying with GetCurrentState.

{"id":"curltest","jsonrpc":"2.0","result":{"blockhash":"8ebedb732eb33f97e49553db1fbfcddbaeedae9ecaaf41e80de66281ef
in Wonderland":"Hello World!","BOB":"HELLO WORLD!","Crawling Chaos":"...Hello, ALICE!","Wile E. Coyote":"Hello Road Runner!"},"height":613228,"state":"up-to-date"}}

Or prettified:


```
{
   "id": "curltest",
   "jsonrpc": "2.0",
   "result": {
     "blockhash":
"8ebedb732eb33f97e49553db1fbfcddbaeedae9ecaaf41e80de66281ef0a79b9",
     "chain": "main",
     "gameid": "helloworld",
     "gamestate": {
       "ALICE": "",
       "Alice in Wonderland": "Hello World!",
       "BOB": "HELLO WORLD!",
       "Crawling Chaos": "...Hello, ALICE!",
       "Wile E. Coyote": "Hello Road Runner!"
     },
     "height": 613228,
     "state": "up-to-date"
   }
}
```

Looking in our hello daemon, we can see our move output.

In Windows:

In Linux:

6.17 Summary

We wrote a Hello World game on the SpaceXpanse platform and learned about a lot of different moving parts. In particular, we overrode 3 libspex methods:

1. GetInitialStateInternal

1. UpdateState

1. GameStateToJson

We then created a main method where we:

1. Processed command line options into flags (gflags)

1. Did some error checking

1. Started libspex with const int res = spacexpanse::DefaultMain (config, "helloworld", logic);

We compiled libspex in such a way that we can add it to any game we choose, just as we did with Hello World.

We compiled Hello World with libspex embedded in our executable file.

We copied (or installed) all the dependencies for our hello daemon and ran it.

We used a JSON RPC to get the game state from the hello daemon with curl.

We made a move through JSON RPC and the name_update method with spacexpanse-cli.

We checked our move once it was mined into the blockchain, again with curl.

We did a lot! Congratulations! You're well on your way to becoming a SpaceXpanse gaming maven!

7 Hello World CX

7.1 Hello World in C#

This code sample is minimalistic. It's goal is to demonstrate how to wire up an application with the SpaceXpanse platform quickly and easily.

HelloSpaceXpanse is a typical "Hello World!" example. It wires up the SpaceXpanse platform in a Windows forms project.

HelloSpaceXpanse lets you say "Hello World!" on the SpaceXpanse blockchain. That's all it does.

Download the HelloSpaceXpanse project here.

NOTE: The only file that is important for this tutorial is "HelloSpaceXpanse.cs". It would be best to open the solution in Visual Studio, then open the HelloSpaceXpanse.cs form and it's code. Nothing else is needed to understand the basics of wiring up libspex, sending moves and getting data.

Follow along with the tutorial here and you'll be in a solid position to get into more advanced topics. Don't get distracted with other parts of the code.

Get a SpaceXpanse Wallet

You'll need a SpaceXpanse wallet and some ROD.

To start, download and install the latest SpaceXpanse Electron wallet here if you're on Windows, or the the SpaceXpanse QT wallet if you're on Linux or Mac. The wallets automatically run the SpaceXpanse daemon, i.e. spacexpansed. (This tutorial assumes Windows.)

If you run the QT or spacexpansed, you'll need to set flags manually. They're automatically set in the Electron wallet. For more information on flags, see the Daemon Flags tutorial.

Run the wallet or spacexpansed and give it time to fully synchronise.

You should also get some ROD. You can buy ROD at Liquid.com or ask in the Development forum. 1 ROD is lots more than enough to get started.

Inside HelloSpaceXpanse

There are 4 distinct elements inside of HelloSpaceXpanse.

- HelloSpaceXpanse project: This is our "game"

- BitcoinLib project: This is used for RPCs

- SpaceXpanseWrapper project: This wraps the C++ 64-bit libspex statically linked libarary in the SpaceXpanseStateProcessor folder

- SpaceXpanseStateProcessor folder: This has the precompiled libspex binary for Windows, libspacexpansewrap.dll, and all its dependencies

While this tutorial only explains HelloSpaceXpanse, you can find more information about other elements elsewhere in the SpaceXpanse documentation and tutorials.

7.2 Important Considerations

VIDEO Set your SpaceXpanse project to 64 bit

The SpaceXpanseWrapper DLL (libspacexpansewrap.dll) is 64-bit. Consequently, your project MUST exclude 32-bit or explicitly be set as 64-bit. See this HelloSpaceXpanse setting:

Otherwise, you must set your project to be 64-bit (x64).

7.3 Implementing SpaceXpanse Simplified

VIDEO Simplified overview of implementing SpaceXpanse

To implement SpaceXpanse, all you need to do is to is:

- Instantiate SpaceXpanseWraper (1 line of code)

- Connect to SpaceXpanseWrapper (1 line of code)

- Set up a listener thread to receive game states

- Implement game logic in:

+ 3 callbacks

- initialCallbackResult (20 lines of trivial code)

- forwardCallbackResult

- backwardCallbackResult

+ Ancillary game logic

- JSON classes

- Helper methods

- Send moves to the SpaceXpanse blockchain

+ This requires RPCs (can be implemented in many ways)

7.4 Threading

It's crucial that you create threads for SpaceXpanseWrapper.

Portions of SpaceXpanseWrapper are blocking operations and MUST be run in separate threads. Specifically, the SpaceXpanseWrapper.spacexpanseGameService.WaitForChange and SpaceXpanseWrapper.Connect methods.

Threading in HelloSpaceXpanse

In HelloSpaceXpanse, we've used BackgroundWorkers. There are more robust threading patterns available, but BackgroundWorkers are simple to understand with little complexity.

You can implement better threading structures on your own.

7.5 Instantiate and Connect to SpaceXpanseWrapper

VIDEO Instantiate and Connect to SpaceXpanseWrapper

Instantiating and Connecting to SpaceXpanseWrapper must be done in a thread. The connection is a blocking operation.

However, once connected, SpaceXpanseWrapper will immediately begin sending log data to the console and game state data to the listener, i.e. through calling the SpaceXpanseWrapper.spacexpanseGameService.GetCurrentState method. The log data isn't game state data though; that is examined below.

To instantiate the wrapper, call it's constructor:

wrapper = new SpaceXpanseWrapper(dataPath, // The path to the game's executable file.

Properties.Settings.Default.Host, // The host, e.g. localhost or 127.0.0.1

"8900", // The game host port. Can be any free port.

ref result, // An error or success message.

CallbackFunctions.initialCallbackResult,

CallbackFunctions.forwardCallbackResult,

CallbackFunctions.backwardCallbackResult);

To connect, call the Connect method:

result = wrapper.Connect(dataPath, // The path to the game's executable file.

FLAGS_spacexpanse_rpc_url, // The URL for RPC calls.

"8900", // The game host port. Can be any free port.

"0", // Which network to use: Mainnet, Testnet, or Regtestnet.

"memory", // The storage type: memory, sqlite, or lmdb.

"helloworld", // The name of the game in the 'g/' namespace.

dataPath + "\\..\\SpaceXpanseStateProcessor\\database\\", // Path to the database folder, e.g. SQLite.

dataPath + "\\..\\SpaceXpanseStateProcessor\\glogs\\"); // Path to glog output folder.

7.6 Listening for New GameStates

VIDEO Listening for new GameStates

We must listen for updates in a thread. There are 4 important lines of code.

In our listener, we use the wrapper to get game states. The first thing to do is to tell the wrapper to wait until there's a new game state.

wrapper.spacexpanseGameService.WaitForChange();

Once there's a game state, the thread resumes and we get the game state.

BitcoinLib.Responses.GameStateResult actualState =

wrapper.spacexpanseGameService.GetCurrentState();

We then cast that as a GameState by deserialising the JSON.

state = JsonConvert.DeserializeObject<GameState>(actualState.gamestate);

The final step in our listening thread is to send the game state to the main UI thread.

sendingWorker.ReportProgress(0, state);

7.7 Update the UI with the New GameState

VIDEO Update the UI with the New GameState

Our listener thread casts the event argument as a GameState and sends it to a method that updates the game.

UpdateHelloChat((GameState)e.UserState);

HelloSpaceXpanse merely updates a text box with what other people have said. It loops through all players and stores data in a StringBuilder.

sb.AppendLine(v.Key + " said \"" + v.Value.hello + "\"");

We then update the textbox in the UI.

txtHelloGameState.Text = sb.ToString();

7.8 Sending Moves to the Blockchain

VIDEO Sending Moves to the Blockchain

You don't need to be running an instance of a game to send moves.

Moves can be sent arbitrarily in many ways. Here are some ways:

- From spacexpanse-cli

- From the SpaceXpanse QT console

- Sending to spacexpansed

- Etc.

When processing moves, you must guard against invalid moves with robust error checking. See the code for example error checks.

In HelloSpaceXpanse, we build the move in a string then send it:

string hello = "{\"g\":{\"helloworld\":{\"m\":\"" + txtHelloWorld.Text + "\"}}}";

spacexpanseService.NameUpdate(this.cbxNames.GetItemText(this.cbxNames.SelectedItem),

hello,

new object());

<!-- # Game Logic

As mentioned above, game logic is handled similar to the following:

- 3 callbacks

- initialCallbackResult

- forwardCallbackResult

- backwardCallbackResult

- Ancillary game logic

- JSON classes

- Helper methods

THIS IS A STUB. THIS PORTION OF THE HELLO WORLD TUTORIAL WILL FOLLOW. -->

Done!

Try running HelloSpaceXpanse and then say hello to everyone!

8 Mover

8.1 Mover Sample Game Overview

If you haven't already, make certain to download or clone the libspex repository. You will need to build libspex from source. See the libspex README here for how to build it.

Before proceeding, you should read the Mover README document.

The following tutorial is not a code walk through. It is a high-level overview of some of the most important elements and concepts that the Mover sample game demonstrates. You may wish to open and read the code when it's being described below.

8.2 Using Mover as a Template

Depending on the complexity of your game, you may wish to use Mover a template for your game and simply replace the game logic with your own.

Mover is a Live MMOG and Practically Unhackable

While Mover is very simple, it should be pointed out that it is a live game that anyone can write a client for, and it is a massively multiplayer online game. It can accommodate many thousands of players.

Undoubtably some people may try to break Mover, but if you code your version of Mover properly, those attempts to break it will be meaningless. The only thing needed is decent error checking.

Blockchain games, such as Mover, are resistant to hacking attempts because moves are entered into the blockchain. This requires using blockchain public-private key encryption, and breaking that encryption simply isn't feasible. Consequently the only issue for moves is to check whether they are valid or not. If they are valid, then they are processed. If they are invalid, then they are ignored.

Despite the simplicity of Mover, the same basic principles apply to any other game written the same way. So, while these claims may seem bold for such a simple game, they hold true at any scale.

Are you ready for this adventure?

8.3 Mover Gameplay

In Mover, players use a SpaceXpanse name to join the game. The first move is always the same for everyone: the player is placed on the 2D-map at the origin, (0, 0).

Players can then enter moves which consist of:

- 1 of 8 possible directions

- A number of steps to take in that direction

The 2D-map is extends infinitely in both the x and y axis.

Players can enter new moves at any time. Moves are sent to the SpaceXpanse blockchain, and then read back into the game where they are processed and the game state is updated.

Any step in any direction is considered equal, i.e. 1 step of a horizontal or vertical move is considered equivalent to 1 step travelling along a diagonal.

As you can see, Mover is a very simple game with very simple logic. This makes examining the sample game and code easier and we can focus on how games can run on the SpaceXpanse platform instead of complex game logic.

8.4 Getting Started with Mover

Now that you have downloaded or cloned the code, look in the mover folder. There are several files there. Of interest to us are these files:

- logic.hpp

- logic.cpp

- main.cpp

In there we'll find libspex used, and the game logic for Mover.

logic.hpp

The logic.hpp file is simple enough. It contains some definitions for methods that we'll create in logic.cpp. The most important methods that we want to examine are:

- GetInitialState

- ProcessForward

- ProcessBackwards

However, do take note of the includes. The includes for libspex are:

#include "spacexpansegame/gamelogic.hpp"

#include "spacexpansegame/storage.hpp"

In your project, you will likely need to include these headers similar to the following:

#include <spacexpansegame/gamelogic.hpp>

#include <spacexpansegame/storage.hpp>

logic.cpp

There's quite a bit in the logic.cpp file, but of most interest to us are the callbacks that implement the game logic.

- GetInitialState: Sets the chain and block height for the game

- ProcessForward: Processes all players moves and updates the game state

- ProcessBackwards: Rewinds the game in case there's a fork or reorg

GetInitialState

The game logic in GetInitialState sets the chain that the game runs on as well as the block height where the game begins. In your game, you'll likely want to start on testnet or regtestnet where you can instantly mine new blocks. Regtestnet is perfect for testing. Remember to include the proper hash for the block it begins on.

ProcessForward

The game logic in ProcessForward is the main game logic. This is where you get 3 critical pieces of information:

- The current game state (the game as it is right now)

- The block data with new moves in it

- Undo information that you need to update

For your game, with that information, you need to take the current game state and update it with the new moves you've received through the blockchain, and process that data to come up with a new game state, i.e. you process the game forward by 1 move.

At the same time, you must create some undo data.

Undo Data

Undo data is a set of data that allows you to rewind a game to a previous state. But, why is this needed?

Time and order on a blockchain differs from our every day conception of time and order. Consequently, on most blockchains inevitably there are some events that need to be dealt with, e.g. reorgs.

Events such as reorgs are extremely difficult to deal with. Luckily, libspex takes care of all the heavy lifting here. However, in order to do that heavy lifting, game developers must keep an inventory of undo data. How you manage this is largely up to you, e.g. database or flat files, but it must be done.

Take note how the undo data is created in the Mover example as you'll need to implement something similar. Mover keeps this in memory, but you will likely wish to commit it to a database, such as SQLite, or other storage system.

With undo data, you must be able to compute the inverse of the state transition, i.e. given a new state and undo data, compute the corresponding old state.

A cheap hack is to return the old state as the undo data. However, this won't be very efficient as you'll need to store orders of magnitude more data. That is, processing undo data lets you use far less drive storage space. Still, it should be sufficient for simple games or for getting started quickly. For an example of this, see here.

Returning Data

Once you've processed the new moves and created an updated game state, you must return data so that you can update your front-end GUI for your players.

Note the ProcessForward signature and how arguments are passed in, and the last few lines in the method. While ProcessForward returns data, it also updates data values for arguments passed by reference. This allows multiple variable to be updates easily for consumption in the front-end GUI.

ProcessBackwards

The ProcessBackwards callback is similar to the ProcessForward callback in some ways. However, its purpose is to rewind the game state by 1 block.

In ProcessForward, we produced undo data. Here we consume that undo data.

The "rewind" logic for Mover is nice and simple. Follow how the undo data is consumed, and how the previous game state is reconstructed.

The only return value here is the rewound game state. In the event that you need to rewind further, you'll need your stored undo data.

Compiling and Running Mover

See Makefile.am in the mover folder.

Summary

We took a high-level overview of the Mover sample game and went over the purpose and methodology of some of the most important elements and concepts.

9 Prerequisites

9.1 Prerequisites

In order to develop games using the SpaceXpanse platform you'll need a SpaceXpanse wallet or spacexpansed and some ROD (mainnet, testnet, or regtestnet ROD are all fine). You'll also need to have your wallet properly configured.

You will also need libspex. This is part of the Game State Processor (GSP). See below for more information.

- Wallets

- ROD

- libspex

- RPC and JSON libraries

9.2 Get a Wallet

There are 4 SpaceXpanse wallet distributions.

1. SpaceXpanse Electron wallet for Windows

2. SpaceXpanse QT wallet for Windows

3. SpaceXpanse QT wallet for Linux

4. SpaceXpanse QT wallet for Mac OS X

When you first run them, they need to download and sync the blockchain. This can take some time and largely depends on your network connection.

About the Electron Wallet

The SpaceXpanse Electron wallet is designed for gaming. Out-of-the-box it comes preconfigured for games with all the proper flags. This is the easiest option.

Further, the Electron wallet contains 2 distinct wallets:

- game.dat: Has no password. Meant for storing smaller amounts of ROD to play games

- vault.dat: Is password protected. Meant for storing larger amounts of ROD

Refer to the Wallets section of the SpaceXpanse Electron Help documentation for a more in depth description.

However, even if you do choose to use the Electron wallet, you should still download the QT wallet as it has some **important tools** that you'll enjoy using.

About the QT Wallets

The QT wallets are all standard wallets and very similar to the Bitcoin Core QT wallet. There aren't any significant differences between the Windows, Linux, and Mac OS X distributions.

If you choose to use a QT wallet during development, you can refer to the Bitcoin Core help for those parts that are common to both SpaceXpanse and Bitcoin.

You can also get help through the SpaceXpanse QT console by typing "help" or "help <command>".

Startup Flags

However, when you run the QT wallet, you must set several flags or you won't be able to connect to it. Run the QT wallet or spacexpansed with these flags (note that datadir is optional):

- -wallet=vault.dat: Loads the vault.dat wallet

- -wallet=game.dat: Loads the game.dat wallet

- -server: Sets spacexpansed to run as a server

- -rpcallowip=127.0.0.1: The IP to allow RPC calls on

- -datadir=<DataDirPath>: OPTIONAL Changes the default data directory to the one specified

- -zmqpubhashtx=tcp://127.0.0.1:28332: See below

- -zmqpubhashblock=tcp://127.0.0.1:28332: See below

- -zmqpubrawblock=tcp://127.0.0.1:28332: See below

- -zmqpubrawtx=tcp://127.0.0.1:28332: See below

- -zmqpubgameblocks=tcp://127.0.0.1:28332: See below

For information about the ZeroMQ flags, refer to the Bitcoin documentation on it here.

By default, the wallet runs on mainnet. If you wish to use testnet or regtestnet, you can use 1 of these flags (but not both).

- -testnet

- -regtest

spacexpanse.conf

You can also set those flags in the spacexpanse.conf file. It resides in the SpaceXpanse data directory. You can read more about the data directory in this Bitcoin documentation; the only difference is changing "Bitcoin" to "SpaceXpanse" in the path.

The configurations are the same as above, except you remove the "-" (dash). Use 1 per line.

SpaceXpanse QT Toolkit

The SpaceXpanse QT wallet download includes useful tools that will make your life much easier and more enjoyable.

The SpaceXpanse Command Line Interface (CLI), spacexpanse-cli, is very useful for running commands quickly. You can think of this as a more convenient version of the QT wallet's console.

Here's a quick tutorial on spacexpanse-cli that you may wish to read later.

About spacexpansed

spacexpansed is the core daemon and where the "work" is done. The QT wallet is just a convenient GUI for it.

9.3 Get Some ROD

You can develop your game using testnet or regtestnet, in which case you don't need to purchase any real ROD.

For testnet ROD, you can mine some yourself or you can contact us to send you some.

For regtestnet ROD, you must mine this yourself. Find out more about regtestnet here.

Both testnet and regtestnet are free, but at some point you will want to run your game on mainnet. You can purchase ROD on an exchange. See the SpaceXpanse website for more information.

9.4 libspex

The libspex daemon does much of the heavy lifting that blockchain programming requires. There are several places to look for it. Click any of the following for what you need. If you have questions, you can check a tutorial or post in the Development forum.

- libspex: The C++ source code

- libspex_wrapper: A wrapper for libspex. Creates a statically linked library for Windows

- Precompiled binaries for Windows: Statically linked library with all its dependencies

- SpaceXpanseWrapper: A C# project that wraps the libspex DLL

You don't need to decide on this right now, but you will need to download whichever option you need when it comes time to start coding.

For information on GSPs and how libspex relates to the SpaceXpanse daemon and your game, see libspex Component Relationships

9.5 RPC and JSON libraries

Creating a game with SpaceXpanse involves a great deal of RPC calls and using JSON. You may wish to use existing libraries rather than roll your own. Whichever you choose is entirely up to you.

10 Build Spacexpansed

10.1 Build spacexpansed

Follow the instructions below to build spacexpansed from source.

Update and Upgrade Ubuntu

Open a terminal and issue the following commands.

sudo apt-get update

sudo apt-get upgrade

Install Dependencies

Issue the following command to install dependencies.

sudo apt-get install git build-essential libtool autotools-dev automake pkg-config libssl-dev libevent-dev bsdmainutils python3 libboost-system-dev libboost-filesystem-dev libboost-chrono-dev libboost-test-dev libboost-thread-dev

Install libdb4.8 (Berkeley DB)

If getting errors during compilation, change '$PWD' with your actual path.

git clone https://github.com/SpaceXpanse/rod-core-wallet

cd rod-core-wallet

chmod +x ./contrib/install_db4.sh

./contrib/install_db4.sh pwd

export BDB_PREFIX='$PWD/db4'

Build spacexpansed

Next, build and install spacexpansed with the commands below.

Regarding the --without-gui --without-miniupnpc, it makes spacexpansed without GUI and ability to automatically open ports on the router, but not is necessary in most cases when compiling on Ubuntu 20.XX. If you want GUI on Linux, please use Ubuntu 18.XX for now.

Regarding make -j"$(($(nproc)-1))", it's trying identify the max number of cores to use to build spacexpansed. Choose "2" or higher number to build spacexpansed faster if that's not working for you.

./autogen.sh

./configure BDB_LIBS="-L${BDB_PREFIX}/lib -ldb_cxx-4.8" BDB_CFLAGS="-I${BDB_PREFIX}/include" --without-gui --without-miniupnpc

make -j"$(($(nproc)-1))"

sudo make install

10.2 CONGRATULATIONS!

You just built spacexpansed!

You can now run and use:

- spacexpanse-qt

- spacexpanse-cli

- spacexpansed

See here for how to run spacexpansed with options.

11 spacexpanse.conf

11.1 spacexpanse.conf

The spacexpanse.conf file in the data directory lets you set default options for spacexpansed.

These options are set whether you run spacexpansed directly, run the SpaceXpanse QT wallet, or run the SpaceXpanse Electron wallet.

You can find an example spacexpanse.conf file in the SpaceXpanse Core respository here or here and a different (older) one here.

You can get a complete list of options with descriptions by running this command:

spacexpansed --help >spacexpansedhelp.txt

Use them as described in that help or as described in the example spacexpanse.conf file.

The majority of options are identical to those used in Bitcoin and Namecoin. However, there are options that are unique to SpaceXpanse.

See the Daemon Options document for a list of differences, e.g. name_register, name_update, etc.

See Also

See more information about running spacexpansed for games here.

See more information about daemon startup options for spacexpansed here.

See more information about spacexpanse-cli here.

See more information about SpaceXpanse RPC methods here.

12 Daemon Options

12.1 Daemon Options

For a full list of daemon startup options, open a command line or terminal and run:

spacexpansed -?

You can also consult the Bitcoin documentation here.

The following documents the most important differences between daemon startup options for spacexpansed and bitcoind.

For standard startup options needed for SpaceXpanse, consult Running spacexpansed for Games here.

12.2 Where to Use Daemon Options

Daemon options are used to start spacexpansed. You can set them:

- In a shortcut to spacexpansed

- In the spacexpanse.conf file in the data directory

- In command line arguments when running spacexpansed

You can also start the SpaceXpanse QT wallet or SpaceXpanse Electron wallet the same way as above. They will pass the options to spacexpansed when it starts up.

12.3 Unique to SpaceXpanse

There are several options that are unique to SpaceXpanse and spacexpansed.

Options

- -namehistory: Keep track of the full name history (default: 0)

The namehistory option allows getting the entire history for a name. However, this increases the database size a little bit. You can then use name_history <name> in an RPC or in spacexpanse-cli to get the complete history for that name.

ZeroMQ Notification Options

- -trackgame=<game>: Enable tracking of the listed game for the SpaceXpanse game interface

- -zmqpubgameblocks=<address>: Enable publication of game data for block attach/detach events in <address>

RPC Server Options

- -maxgameblockattaches=<n>: Sets the maximum number of attach steps sent for a single game_sendupdates request (default: 1000)

- -nameencoding=<enc>: Sets the default encoding used for names in the RPC interface (default: utf8)

- -valueencoding=<enc>: Sets the default encoding used for values in the RPC interface (default: ascii)

Debugging/Testing options

- -debug=<category>

  Output debugging information (default: -nodebug, supplying <category> is

  optional). If <category> is not supplied or if <category> = 1,

  output all debugging information. <category> can be: net, tor,

  mempool, http, bench, zmq, db, rpc, estimatefee, addrman,

  selectcoins, reindex, cmpctblock, rand, prune, proxy, mempoolrej,

  libevent, coindb, qt, leveldb, names, game.

The "names" and "game" categories are added to the debug option in SpaceXpanse, but are not found in Bitcoin.

Block creation options

- -blockmaxweight=<n>: Set maximum BIP141 block weight (default: 396000)

The maximum in Bitcoin is 3996000. SpaceXpanse has smaller, faster blocks, so hence the difference.

Wallet options

- -addresstype: What type of addresses to use ("legacy", "p2sh-segwit", or "bech32", default: "legacy")

The default in SpaceXpanse is "legacy", however the default in Bitcoin is "p2sh-segwit".

Ports

- -port=<port>: Listen for connections on <port> (default: 11998, testnet: 18398, signet: 38398, regtest: 18498)
- -rpcport=<port>: Listen for JSON-RPC connections on <port> (default: 11999, testnet: 18399, signet: 38399, regtest: 18499)

Port numbers differ between SpaceXpanse and Bitcoin.

See Also

See more information about spacexpanse.conf here.

See more information about spacexpanse-cli here.

See more information about SpaceXpanse RPC methods here.

See more information about Running spacexpansed for Games here.

13 Spacexpanse Cli

13.1 Getting Started with spacexpanse-cli

The SpaceXpanse Command Line Interface (spacexpanse-cli) program is a CLI for interacting with the SpaceXpanse daemon and SpaceXpanse wallets through JSON-RPC.

Much of what can be done in spacexpanse-cli can also be done in the SpaceXpanse QT wallet's console. However, there are advantages to using spacexpanse-cli instead of the QT console. As such, much of the SpaceXpanse documentation uses spacexpanse-cli.

The following is to help you get up to speed on how to use spacexpanse-cli. If you are already comfortable with CLIs, then you can simply run it with the -? or --help parameters to get the information you need and skip this tutorial. That help is the official reference material.

This tutorial is designed as a relaxing walk through that you can follow along with and experiment with as you read. Take time to run some of the commands and see how they work for you. Experience is a great teacher.

13.2 Additional Documentation from Bitcoin

Much of what you will need to know about commands and JSON-RPC methods is already documented in Bitcoin documentation. This tutorial is not a substitute for upstream documentation.

13.3 Getting Ready...

If you haven't already, download the SpaceXpanse QT wallet software here and extract the contents of the ZIP file.

- spacexpanse-cli

- spacexpanse-hash

- spacexpanse-qt

- spacexpanse-tx

- spacexpansed

On Linux you'll need to chmod +x them to make them executable.

13.4 Start Your Wallet or spacexpansed

Here we'll run spacexpansed instead of one of the wallet software packages. Run it with the server command.

spacexpansed -server=1

> For more information about options, see Running spacexpansed for Games here and Daemon Options.

13.5 Let's Get Started!

Open a console (command prompt in Windows or terminal in Linux) and navigate to the spacexpanse-cli folder.

You can get help for spacexpanse-cli options with this command:

spacexpanse-cli -?

Run that now and have a quick glance.

At the top of the help you'll see some usage cases. Here we'll only examine the first one:

> spacexpanse-cli [options] <command> [params]

The [options] are from the help that you just viewed. (The list of commands is available from the SpaceXpanse QT or Electron wallet console by typing "help".) For many commands we don't need to specify any options and can simply send a command with parameters. Here are some examples. They are all are perfectly safe to use, so feel free to try them right now.

spacexpanse-cli getblockcount

spacexpanse-cli getdifficulty

spacexpanse-cli getblockchaininfo

spacexpanse-cli getblockhash 123

spacexpanse-cli gettxoutsetinfo

spacexpanse-cli uptime

spacexpanse-cli game_sendupdates "g/mv" "2aed5640a3be8a2f32cdea68c3d72d7196a7efbfe2cbace34435a3eef97561f2" "3b365d712d87e354a36a6b0445fd022322e559bb9b4dc493f8eea3328d670197"

spacexpanse-cli trackedgames

spacexpanse-cli trackedgames "add" "mv"

Playing Safely

As above, these are safe commands and you won't do any damage by using them. We'll warn you for commands that have real consequences.

To interact with SpaceXpanse with zero fear, you can run regtest. See the Regtestnet tutorial for information about that. For now, we return back to the real world mainnet.

Options vs. Commands

The options we saw above are for spacexpanse-cli. It passes those options on to the SpaceXpanse daemon. Some of the options that you will need most include:

- -testnet

- -regtest

- -rpcwallet=<walletname>

- -rpcuser=<user>

- -rpcpassword=<pw>

Commands

Commands are JSON-RPC methods defined in spacexpansed. You can get a complete list of commands from the console in either the SpaceXpanse QT or Electron wallet by typing "help".

Commands "do things".

Commands can put data onto the blockchain, get data from the blockchain, manage wallets, and much more. Skim through the spacexpansed-help.txt file to get a feel for the kinds of commands that are available to you. Most are the same as those in bitcoind.

Also consult SpaceXpanse RPC Methods for SpaceXpanse-specific RPC methods.

For more in-depth information on specific methods, you must use the SpaceXpanse QT or Electron wallet console. Run "help <command name>". So, for example, if you want information about creating new addresses, use this:

help getnewaddress

You can also check the Bitcoin documentation for commands as much of it is the same.

Continuing on...

Whether or not you need options depends upon what starting options were passed to the wallet software when it started up, or any options that were set inside the spacexpanse.conf file in the data directory. You can find spacexpanse.conf in the data directory here on Windows, Linux, and Mac OS X, respectively:

%appdata%\SpaceXpanse\spacexpanse.conf

~/.spacexpanse/spacexpanse.conf

~/Library/Application Support/SpaceXpanse/spacexpanse.conf

Dealing with Wallets

When working with real wallets, such as the Electron game wallet, we must specify which wallet to use with the -rpcwallet=<walletname> option.

To load a wallet.dat file, specify which one similar to any of the following commands.

spacexpanse-cli loadwallet "wallet.dat"

spacexpanse-cli loadwallet "game.dat"

spacexpanse-cli loadwallet "vault.dat"

The following command gets the balance from the game wallet and vault wallet, respectively.

spacexpanse-cli -rpcwallet=game.dat getbalance

spacexpanse-cli -rpcwallet=vault.dat getbalance

This will not cause the wallet to appear in the Electron UI, but it will make the wallet available to the SpaceXpanse daemon, which is what spacexpanse-cli is querying.

You can get a list of currently loaded wallets using the following command:

spacexpanse-cli listwallets

The result after having loaded the 3 wallets above would be as follows.

[

"vault.dat",

"game.dat",

"wallet.dat"

|

You can unload a wallet as shown below:

spacexpanse-cli unloadwallet "wallet.dat"

13.6 Some Cooler Stuff You Can Do

Now that we've looked a bit at wallets, let's do some cooler stuff starting with backing up a wallet. For this, we must specify the wallet to back up with a spacexpanse-cli option, e.g. "-rpcwallet=game.dat". We must also specify a complete path, e.g. "C:\SpaceXpanse Backups\MyFirstBackup.dat". Make certain that the folder exists.

spacexpanse-cli -rpcwallet=game.dat backupwallet "C:\SpaceXpanse Backups\MyFirstBackup.dat"

Creating a ROD Address

Now that we have a backup of our game wallet, let's do some more cool stuff like creating a new address.

spacexpanse-cli -rpcwallet=game.dat getnewaddress "This is my new address!"

The command will run and then return an address, e.g. "CXTdDBYrZEmSHWrsq8hFv5R78NrH8Q2Bxd".

You can now load your Electron wallet and verify that the address is in there. Check the Receive tab and scroll down. Addresses are listed alphabetically by label. (You'll need to shut down spacexpansed, start Electron, check, close Electron, then run spacexpansed as above if you decide to check and then continue on in this tutorial.)

Any cryptocurrency will let you do that; it's very basic. However, there are methods that you won't find in any other cryptocurrencies, other than perhaps some in Namecoin and Huntercoin. But even those don't have all the options you have in SpaceXpanse.

Registering a Name

NOTE: This operation costs ROD and is permanent on the blockchain.

If your or your user don't already have a name, you can register one for them. For users, you would normally do this inside of your game through the RPC interface. Let's get you a new name! Replace "Bugs Bunny" with the name you'd like to have.

spacexpanse-cli -rpcwallet=game.dat name_register "p/Bugs Bunny" "{}"

The parameters are:

- Name: "p/Bugs Bunny"

- Value: "{}"

The name must have a namespace. For player accounts this is "p/". Everything after that is the "name" per se. Refer to Name and Value Restrictions for more information.

The value must be valid JSON. In this case we simply send an empty value. There's no reason to do more than that if the goal is to simply register a name. However, it could include an initial move in a game. (Refer to Moves for the "move" specification.)

The return value for that command was "1e57a1136711e008f5046025613c1835515eb7adf175a15bab8a52b3eec28e5f". You can verify this on the SpaceXpanse blockchain by using spacexpanse-cli, or you can check the SpaceXpanse blockchain explorer here.

As mentioned above, you can get more help on the name_register operation with help name_register in the QT or Electron console.

List your names

To see all the names in your wallet, use the name_list RPC method. Make sure to specify which wallet you want to check.

spacexpanse-cli -rpcwallet=game.dat name_list

13.7 Summary

We've taken a leisurely journey into examining spacexpanse-cli. We walked through different kinds of examples. In some examples we used options for spacexpanse-cli and in others we didn't need any options. We looked at several generic commands that are also found in Bitcoin. We also looked at some commands that are specific to SpaceXpanse.

The techniques you learned above through examples can be applied more generally. Check the documentation for the options and commands as shown above.

Remember to check the regtestnet tutorial to find out how you can experiment in a private blockchain without fear of making a mistake.

See Also

See more information about SpaceXpanse RPC methods here.

See more information about daemon options for spacexpansed here.

See more information about running spacexpansed for games here.

See more information about spacexpanse.conf here.

14 Downloads

14.1 Downloads

There are several downloads for SpaceXpanse. They include everything from the bare essentials to convenient precompiled binaries, helpful projects, and tutorial downloads. The bare minimum that you need are spacexpansed and libspex.

Some downloads may require you to clone them in GitHub or download the entire repository.

14.2 SpaceXpanse Daemon

The SpaceXpanse daemon (spacexpansed) is packaged in the SpaceXpanse QT wallet download. You can find that here.

In addition to the QT wallet, an Electron wallet is available. It's the primary wallet for end users and is designed to be used with games. It is also packaged with spacexpansed. You can find that here.

14.3 SPV (lite) Wallet

The SpaceXpanse Electrum ROD wallet is an Simple Payment Verification (SPV) wallet or "lite" wallet. It doesn't download the entire blockchain so it is much faster and easier to get up and running with for end users. It is also used in headless mode (no GUI or visible interface) for "Simple Mode" in games running on the SpaceXpanse blockchain, e.g. Treat Fighter and Taurion.

Building a "Simple/Lite" Mode into Your Game

Commands and syntax for the Electrum wallet differ from the regular spacexpansed commands and syntax. You can consult the Electrum official documentation or examine the source code for SpaceXpanse blockchain games that incorporate Electrum for a simple/lite mode, e.g. the front end for Ships.

14.4 Developer Downloads

The following downloads are for developers.

libspex

The libspex library is what you use to build games. It takes care of all the difficult blockchain operations and leaves you free to focus on your game. It's written in C++. You can find it here.

Projects Inside SpaceXpanse Unity Mover

The SpaceXpanse Unity Mover repository contains several packages that you may wish to use.

- SpaceXpanseStateProcessor: This is libspex with all its dependencies precompiled for Windows in the "SpaceXpanseStateProcessor" folder. It's a statically linked C++ library

- SpaceXpanseWrapper: This is a C# wrapper for libspex. You can use it out-of-the-box by compiling it, or you can modify it as you wish

- BitcoinLib: This is a convenient RPC library that has been modified for SpaceXpanse. You can use it out-of-the-box by compiling it, or you can modify it as you wish

It also contains the SpaceXpanseUnity project. This a C# Unity implementation of the Mover game that is bundled with libspex (see above). You can find a tutorial for it here.

You can find all of these available for download in the spexlib_unity GitHub repository here.

Windows Wrapper for libspex

The libspex_wrapper repository is a wrapper for Windows using cmake. For the precompiled binaries, see the SpaceXpanse Unity Mover download above. You can find the libspex_wrapper repository here.

Tutorial Code

Code for tutorials is available in the SpaceXpanse Tutorial Code repository.

Hello World

Code for the Hello World tutorials is available in the SpaceXpanse Tutorial Code repository.

Mover

Code for the Mover tutorials is available in the SpaceXpanse Tutorial Code repository.

The C++ Mover code is part of integrated tests, and is available in the libspex repository.

Ships

The Ships source code demonstrates how to use the sqlitegame class and the SpaceXpanse Side Channels technology for real-time games.

The Ships source code is split into the front end code in its own repository, and the GSP code, which is part of the libspex library.

15 Getting Started With Regtestnet

15.1 Getting Started with Regtestnet

During development you'll need to interact with the SpaceXpanse blockchain. However, using real ROD is expensive and permanent on the blockchain. Using testnet is a much better option, but you will still need to mine testnet ROD to use as testnet mimics mainnet, including how mining is done.

Regtestnet is a better solution for developers looking to put their games and apps on the SpaceXpanse blockchain. It is a private network where no peers are needed and you can mine new blocks at will.

15.2 What is Regtestnet?

Regtestnet is a private blockchain where no peers are needed. It is entirely separate from mainnet and testnet, and you can mine blocks whenever you wish.

Regtestnet is also called:

- Regression test mode

- regtest

- regtest mode

Note that the RPC port numbers differ for the various networks:

- Mainnet = 11999

- Testnet = 18399

- Regtestnet = 18499

As do the P2P ports:

- Mainnet = 11998

- Testnet = 18398

- Regtestnet = 18498

15.3 Back Up Your Wallets

Before doing any kind of development, make sure that you've backed up your wallets. See the SpaceXpanse forums for tutorials and videos about how to do that. Which ever interface you choose to use is entirely up to you.

15.4 SpaceXpanse-cli vs. SpaceXpanse QT Console

The examples here use spacexpanse-cli from a console (command prompt/terminal) instead of using the QT wallet's console.

For those unfamiliar, you can simply delete the "spacexpanse-cli -regtest" from the beginning and the run the rest of the command in the QT console, e.g. the spacexpanse-cli command:

spacexpanse-cli -regtest getbalance

In the QT console would simply become:

getbalance

One major advantage of using spacexpanse-cli is that you can pipe output to a file. e.g.:

spacexpanse-cli -regtest getbalance >mybalance.txt

15.5 Firing Up Regtestnet

To use regtestnet, you'll need to download the SpaceXpanse QT wallet and spacexpanse-cli. If you haven't already, you can download that here. (You still can use spacexpanse-cli with the Electron wallet, which is most convenient when starting with SpaceXpanse.)

Open a command prompt or terminal and navigate to the folder where you extracted the QT files.

Next, run the QT wallet with the -regtest and server=1 options as shown below. You must use the -server=1 option in order to use spacexpanse-cli, otherwise you'll get errors from spacexpanse-cli because it cannot connect.

spacexpanse-qt -regtest -server=1

The SpaceXpanse QT wallet program will open up and you'll have a zero ROD balance. You can ignore the first screen; click the Hide button.

You're now running your own private regtestnet.

Multiple Peers in Your Regtestnet Network

NOTE: You may wish to skip this section and return to it once you've finished this tutorial. Don't get hung up on this to start. This will be most useful to you once you've got your game well into development.

You may wish to have more than a single instance of your game running on regtestnet. To accomplish this, use the addnode command.

You'll need to know your local area network IP addresses, e.g. 192.168.0.123 and 192.168.0.135. If you need to use ports when specifying the IP address, use the P2P port, e.g. 192.168.0.123:18495. If you're running the daemon with a custom spacexpanse.conf file and you've changed ports in there, then you'll need to specify them as per your spacexpanse.conf file entries.

On the "123" machine, run the following command:

spacexpanse-cli -regtest addnode "192.168.0.135" "add"

This will add the "135" machine as a node to the "123" machine.

On the "135" machine, run the following command:

spacexpanse-cli -regtest addnode "192.168.0.123" "add"

This will add the "123" machine as a node to the "135" machine.

NOTE: The return value for addnode is null and getnodeaddresses will return an empty set. This is normal. Instead, run getpeerinfo.

spacexpanse-cli -regtest getpeerinfo

That will return a set of information about peers. Check the addr field to see the IP address of the connected node.

They should sync very quickly, however, you may wish to mine a few blocks. See below for how to do that.

15.6 Check Your ROD Balance

Back in your command prompt or terminal, check your balance with the -regtest option and the getbalance command:

spacexpanse-cli -regtest getbalance

You should see your zero balance as "0.00000000", i.e. to 8 decimal places.

15.7 Create a ROD Address

Next, you'll need to create address so that you can mine regtestnet ROD into. The getnewaddress command takes 2 optional arguments. In this example, we only set the first one for the label. Enter the following command into your command line:

spacexpanse-cli -regtest getnewaddress "My label"

That will output an address for you, e.g. "ceeABTxXdFaeL4eJKrAHqEatXXb7mwk1tS". You can also check that it is in your restestnet wallet in the QT client through Window > Receiving addresses...

15.8 Mine Some ROD

When coins are mined, they are "immature" and cannot be spent until they become "mature" after 100 confirmations.

Consequently, in order to get 1 block reward of coins that are spendable, you must mine at least 101 blocks.

Before continuing, you may wish to set the number of buffers in your command prompt or terminal to a higher number so that you can scroll up. For our purposes here, it should be more than 101. However, this is entirely optional and does not affect anything in regtestnet; it is purely for convenience.

The following command uses the address generated above to mine 101 blocks.

spacexpanse-cli -regtest generatetoaddress 101 ceeABTxXdFaeL4eJKrAHqEatXXb7mwk1tS

This outputs a set of 101 block hashes, e.g.:

[

"7e5ae5e5ba085957dd94e7c6fb0d77847797e85d45e900a99228b3ef91058566",

"eab4868ab84ef0b9f66b2c7fc8105323ddf5f085530d0f6aed5345dc27e7aa74",

...

"3cb0837fcda37070faf455e6143e8fd494b483362707616647a370638f844951"

]

You can run the getbalance command used above to see that you now have 50 ROD in your wallet. Remember, newly mined coins require 100 blocks to mature and be spendable. You can also look in the QT client to see the results.

Here we see the Overview with 50 ROD available, and 5000 ROD immature.

You can also check the Transactions tab and scroll to see that only 1 is fully mature (confirmed).

You can repeat the above generatetoaddress command and you'll see that your balance increases. However, as you mine more blocks, the number of immature coins will slowly decline because immature coins require 100 confirmations to mature and the block reward declines very rapidly compared to mainnet, e.g. by block 1200 the reward will be 0.19531250 ROD.

Mining at Will

As you can see from the above, you can mine new regtestnet blocks/ROD at will. However, unless you specifically mine new ROD, no more blocks will be mined.

Depending upon your requirements, you may wish to create a small program that lets you mine new blocks in a controlled way, e.g. 1 block at a time, or perhaps spurts of 10 or more blocks at once, or perhaps on a timer at specific intervals, e.g. every 30 seconds.

Forcing Reorgs to Test Undo

There are times when a blockchain undergoes a reorg. These situations are difficult to deal with, but that hard work is all done for you through the libspex library.

During the development of your game, you'll need to test your undo data, but this can't realistically be done on mainnet or testnet as you cannot predict when a reorg event will happen. Consequently, you must use regtestnet. You'll need the invalidateblock and reconsiderblock methods to do this.

Your testing workflow will typically follow this pattern:

1. Call the invalidateblock method

2. Do a move in your game

3. Call the generatetoaddress method to mine the move in step #2 onto the blockchain

4. Possibly repeat steps #2 and #3 to get more moves into the blockchain

5. Call the reconsiderblock method

You can then see how your code handles that situation and react accordingly.

The invalidateblock and reconsiderblock methods take a blockhash as their parameters. You can call them as shown below:

```
spacexpanse-cli -regtest invalidateblock "12d102d12ddbe8fe0dabc03e9488efad481d87c0b084398307a7fb54592fb26c"
```

```
spacexpanse-cli -regtest reconsiderblock "12d102d12ddbe8fe0dabc03e9488efad481d87c0b084398307a7fb54592fb26c"
```

Resetting Regtestnet

To reset regtestnet, simply rename or delete the regtest folder in the data directory.

15.9 Summary

In this tutorial we looked at:

- How to start regtestnet

+ How to add nodes (other machines) to regtestnet

- How to get a balance in the wallet

- How to create a ROD address

- How to mine new blocks and get regtestnet ROD

- How to force reorgs to test undo data

15.10 Where to Go from Here?

There are other tutorials, but perhaps what would be most useful at this point is for you to create a regtestnet program that lets you mine new blocks in a controlled way. Using spacexpanse-cli may be an option for you if you like using consoles. However, you may wish to create a program with a GUI for convenience. If so, you'll most likely want to use the RPC interface to issue commands directly to the daemon. There are 2 RPC tutorials to help you get started there:

- SpaceXpanse RPC Methods

- RPC Windows C# Tutorial

The RPC methods are the same ones used above as spacexpanse-cli commands.

16 Ships

16.1 Ships

The following documents how to get Ships up and running and how to play the game. The basic steps are:

1. Get a SpaceXpanse Electron wallet

1. Get some ROD

1. Download and run Ships

1. Create or join a Ships Side Channel

1. Play Ships

1. SpaceXpanse Wallet

The first thing you need is the SpaceXpanse Electron wallet. Download it from the SpaceXpanse Download page here:

https://github.com/SpaceXpanse/rod-core-wallet/releases

When you install the wallet, there's an option to download the blockchain. This will significantly accelerate the blockchain synchronization. Choose "Yes" if your blockchain isn't already sync'd. The blockchain will then quickly synchronize.

2. ROD

You need a tiny amount of ROD to buy your SpaceXpanse name and pay transactions fees when you open/close a Side Channel. If you don't have any ROD, go to our Ships Discord channel and ask for some. Make sure to paste in your ROD address from the Electron wallet that you got above. An admin will respond as soon as possible and send you some. The Ships Discord channel is here:

https://discord.gg/uPJdncgrBe

3. Download and Run Ships

Go to the Ships page and download the latest build from there.

https://github.com/SpaceXpanse/libspex/releases

Unzip the archive to a new folder then double-click the "Ships.exe" file to run Ships. You must wait for it to synchronise. You can see that in the top menu bar beside "State". Also, see #4 in the diagram in step 4 below.

The first screen if to choose a screen resolution. Check "Windowed" and choose a resolution then click "Play!".

On the next screen, check the "Run GSP Locally" checkbox. This is critically important as it runs the actual game, which is separate from the lovely GUI. Next, click the Submit button.

On the next screen, choose a username under "select account" then click "go", or enter a new name and click "submit".

The game then starts.

4. Create or Join a Ships Side Channel

The Ships game is played in Side Channels. You must either create one and wait for an opponent to join, or you can join an existing one.

To create a Side Channel:

1. Click the LOBBY menu (1)

1. Click CREATE CHANNEL+ (2)

1. Done

You must then wait for an opponent to join.

To join an existing channel:

1. Click the LOBBY menu (1)

1. In the lobby, click a green Join button to join that channel (3)

1. Done

You're now playing Ships with an opponent!

NOTE: Creating, closing, and joining a Side Channel takes 1 block.

5. Play Ships

To start, both players must drag ships onto their board. Double-click a ship to rotate it.

When you are finished placing your ships, click the Submit Position button in the upper-right corner.

When it is your turn, you'll see "YOUR TURN!" beside a blue circle in the upper-right corner (5). Double-click on an empty square in the red grid to fire a shot at your opponent (6).

Continue back and forth until one player's entire fleet has been sunk.

The Side Channel will automatically close when the game is over. It takes 1 block for the channel to close. A victory (or defeat) message is then displayed.

17 FAQs

Some questions and answers here.

18 How To Get RODs

There are several ways to obtain ROD coins, depending on your preferences and goals. Some of the most common methods are:

I. Mining

- You can mine them SOLO against the SpaceXpanse ROD wallet

You can mine ROD solo with the following miners:

NVIDIA Win64 Miner

You just have to follow the instructions in Readme1st.txt: just add your ROD address to the corresponding .bat file and then start it.

P.S. In the beginning☒ it was possible to mine them with CPU by using generatetoaddress 100 YOURRODADDRESS 1000000000 in the wallet's console, where 100 is the number of blocks to be generated and 1000000000 is the number of iterations. It's only possible to find a block that way if you have a really powerful CPU and, preferably, untethered source of electricity. You can also change the algo by adding sha256d at the end of it but now it's virtually impossible to find block that way☒.

- You can mine them at POOL through well known mining software

You can mine ROD at pools with the following miners:

NVIDIA Win64 Miner

AMD Win64 and Linux64 Miner

You just have to follow the instructions in Readme1st.txt: just add your ROD address to the corresponding .bat file and then start it.

- You can mine them at POOL as secondary coin while mining other coins ([merged mining](https://academy.binance.com/en/glossary/merged-mining)) with dedicated hardware (ASICs)

It's an advanced method and requires specific hardware, so if you don't know how to do it, better stick to the others.

To find available pools, follow this link.

II. Buying

- Buy ROD coins directly from a cryptocurrency exchange

You will need to create an account, maybe verify your identity, and fund your account with fiat currency or another crypto coin. Then you can choose how much ROD coins you want to buy and place an order.

To find available crypto exchanges, follow this link.

III. Get them for FREE

- Using a crypto faucet that rewards you with small amounts of crypto coins for completing tasks, such as watching ads, playing games, or filling out surveys. You will need to have a crypto wallet address to receive the coins. You will also need to be patient, as faucets usually pay very little and have withdrawal limits.

https://graviex.net/faucets/list - look for ROD

https://xeggex.com/faucet - look for ROD

https://faucet.mecrypto.club/ROD/ - only miners can claim

IV. Using liquidity pools and/or atomic swaps

- Liquidity Pools work similar to how pools work on DEX platforms such as Pancakeswap and Uniswap.

Users who add liquidity to a pool are called 'liquidity providers' and they must provide liquidity in both assets. Liquidity providers earn a portion of any trade fees collected from trades in the pool. If a liquidity provider is providing 50% of the liquidity in the pool, then they will earn 50% of the trade fees collected on every trade in the pool. You may remove your liquidity at any time. The prices paid by traders in a pool are based solely upon the liquidity in the pool. This is commonly referred to as 'Automated Market Making'.

For now you can do this at Xeggex liquidity pools and their swap function when become available again.

☒Until difficulty risen enough to be almost impossible to find a block that way.

19 Running Spacexpansed For Games

19.1 Running spacexpansed for Games

It's important to set the right options when you run spacexpansed so that you can communicate with it and so that libspex can also communicate with it.

The short answer is that you should generally run it as follows.

spacexpansed -rpcuser=user -rpcpassword=password -wallet=game.dat -server=1 -rpcallowip=127.0.0.1 -zmqpubgameblocks=tcp://127.0.0.1:28332 -zmqpubgamepending=tcp://127.0.0.1:28332

It will then run as a server.

However, you will need to supply the rpcuser ("user" above), the rpcpassword ("password" above), and the wallet for every request you make. Using spacexpanse-cli a command would then look like this:

spacexpanse-cli -rpcuser=user -rpcpassword=password -wallet=game.dat <command>

If you have specified the rpcuser, rpcpassword or wallet in spacexpanse.conf in the data directory, then you can leave them out above when running spacexpansed. You will then need to specify the rpcuser, rpcpassword and wallet from the spacexpanse.conf file when you issue commands with spacexpanse-cli or through any other RPC tool that you may use.

If you do not specify a wallet anywhere, spacexpansed will default to the wallet.dat file in the data directory.

Running Naked

You can run spacexpansed without specifying any rpcuser, rpcpassword or wallet, in which case you can leave them out entirely when you run commands through spacexpanse-cli. spacexpanse-cli commands are then much simpler.

spacexpanse-cli <command>

To run spacexpansed very simple for spacexpanse-cli and not specify any rpcuser, rpcpassword or wallet in spacexpanse.conf, run spacexpansed like so:

spacexpansed -server=1 -rpcallowip=127.0.0.1 -zmqpubhashtx=tcp://127.0.0.1:28332 -zmqpubhashblock=tcp://127.0.0.1:28332 -zmqpubrawblock=tcp://127.0.0.1:28332 -zmqpubrawtx=tcp://127.0.0.1:28332 -zmqpubgameblocks=tcp://127.0.0.1:28332

That is the minimum you must use to run spacexpansed for games, testing, or development.

See Also

See more information about spacexpanse.conf here.

See more information about spacexpanse-cli here.

See more information about SpaceXpanse RPC methods here.

See more information about daemon startup options for spacexpansed here.

20 SpaceXpanseWrapper

20.1 SpaceXpanseWrapper

This is a C# wrapper for the libspex pre-compiled binaries for Windows.

The statically linked C++ pre-compiled binaries for Windows (SpaceXpanseStateProcessor) and SpaceXpanseWrapper are both available in the spexlib_unity repository.